

DESIGN AND IMPLEMENTATION OF A WHITEBOARD DRAWING SYSTEM
FOR THE TIAGO MOBILE MANIPULATOR

A THESIS SUBMITTED TO
THE GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES
OF
MIDDLE EAST TECHNICAL UNIVERSITY

BY

ANIL GÜLBEDEN

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF MASTER OF SCIENCE
IN
ELECTRICAL AND ELECTRONICS ENGINEERING

SEPTEMBER 2025

Approval of the thesis:

**DESIGN AND IMPLEMENTATION OF A WHITEBOARD DRAWING
SYSTEM FOR THE TIAGO MOBILE MANIPULATOR**

submitted by **ANIL GÜLBEDEN** in partial fulfillment of the requirements for the
degree of **Master of Science in Electrical and Electronics Engineering Department,**
Middle East Technical University by,

Prof. Dr. Naci Emre Altun
Dean, **Graduate School of Natural and Applied Sciences** _____

Prof. Dr. Nevzat Güneri Gençer
Head of Department, **Electrical and Electronics Engineering** _____

Prof. Dr. Afşar Saranlı
Supervisor, **Electrical and Electronics Engineering, METU** _____

Assoc. Prof. Dr. Ahmet Buğra Koku
Co-supervisor, **Mechanical Engineering, METU** _____

Examining Committee Members:

Assoc. Prof. Dr. Mustafa Mert Ankaralı
Electrical and Electronics Engineering, METU _____

Prof. Dr. Afşar Saranlı
Electrical and Electronics Engineering, METU _____

Assoc. Prof. Dr. İsmail Uyanık
Electrical and Electronics Engineering, Hacettepe University _____

Date:01.09.2025

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Name, Surname: Anıl Gülbeden

Signature :

ABSTRACT

DESIGN AND IMPLEMENTATION OF A WHITEBOARD DRAWING SYSTEM FOR THE TIAGO MOBILE MANIPULATOR

Gülbeden, Anıl

Master of Science, Department of Electrical and Electronics Engineering

Supervisor: Prof. Dr. Afşar Saranlı

Co-Supervisor: Assoc. Prof. Dr. Ahmet Buğra Koku

September 2025, 85 pages

Most methods of communication with a robot today involve digital interfaces. While these methods are effective enough to exchange information, humans can use various other means to communicate with each other. In this thesis, we explore a medium of communication underutilized in robotics: drawing. We design and implement a software system for a mobile manipulator to give the robot the capability to draw on whiteboards. We evaluate different methods for each stage of the drawing system for their performance and reliability. We determine a combination of methods that form a high-reliability algorithm that can detect and move to whiteboards, and draw arbitrary trajectories on the whiteboard. We present experimental results on various parts of this problem.

Keywords: mobile manipulator, robotic drawing, robot motion planning

ÖZ

TIAGO MOBİL MANİPÜLATÖRÜ İÇİN BEYAZ TAHTAYA ÇİZİM YAPABİLEN BİR SİSTEMİN TASARIMI VE GERÇEKLEŞTİRİLMESİ

Gülbeden, Anıl

Yüksek Lisans, Elektrik ve Elektronik Mühendisliği Bölümü

Tez Yöneticisi: Prof. Dr. Afşar Saranlı

Ortak Tez Yöneticisi: Doç. Dr. Ahmet Buğra Koku

Eylül 2025 , 85 sayfa

Günümüzde robotlarla iletişim kurmak için kullanılan çoğu yol dijital arayüzlere dayanmaktadır. Bu yöntemler bilgi alışverişi için yeterli olsa da, insanlar birbirleriyle iletişim kurmak için çeşitli diğer araçları kullanabilirler. Bu tezde, robotikte az kullanılan bir iletişim aracı olan çizimi araştırıyoruz. Mobil bir manipülatör için robota beyaz tahtalara çizim yapma yeteneği kazandırmak üzere bir yazılım sistemi tasarlayıp gerçekleştiriyoruz. Çizim sisteminin her aşaması için farklı yöntemleri performans ve güvenilirlik açısından değerlendiriyoruz. Beyaz tahtalara gidip tahtaya istenenleri çizebilen, yüksek güvenilirliğe sahip bir yöntem kombinasyonunu belirliyoruz. Problemin farklı kısımları için deneysel sonuçlar sunuyoruz.

Anahtar Kelimeler: mobil manipülatör, robotik çizim, gezinge planlama

To Mom and Dad

ACKNOWLEDGMENTS

I wish to thank my supervisors Prof. Dr. Afşar Saranlı and Assoc. Prof. Dr. Ahmet Buğra Koku for their support, advice and encouragement. Their invaluable guidance helped me change course whenever I got stuck on a tough problem and motivated me to press on.

I would also like to express my gratitude to the ROMER and RoLab laboratories for providing me with the equipment and workspace that made my thesis possible.

Many thanks to my friends at ROMER and RoLab, whose suggestions, feedback and help greatly improved my thesis. Special thanks go to my labmates, Umut Kurt, Denge Uzel and Umutcan Üstündaş.

This work was supported by TÜBİTAK under the 2210-A Scholarship Program. I extend my thanks to them for their financial support through this endeavour.

Finally, I would like to thank my parents for their unwavering support and love.

TABLE OF CONTENTS

ABSTRACT	v
ÖZ	vi
ACKNOWLEDGMENTS	viii
TABLE OF CONTENTS	ix
LIST OF TABLES	xii
LIST OF FIGURES	xiii
LIST OF ABBREVIATIONS	xix
CHAPTERS	
1 INTRODUCTION	1
1.1 Motivation and Problem Definition	1
1.2 Proposed Methods and Models	2
1.3 Contributions and Novelties	2
1.4 The Outline of the Thesis	3
2 LITERATURE REVIEW	5
3 SUBPROBLEMS IN MOBILE MANIPULATION	7
3.1 System Overview	7
3.2 Board Detection	9
3.2.1 Depth-based Board Pose Estimation	11

3.2.2	RGB-based Mask Estimation	12
3.2.3	Marker-based Board Detection and Pose Estimation	14
3.3	Depth Camera Calibration	15
3.3.1	Distortion Model and Correction Algorithm	17
3.3.2	Calibration Methods	19
3.4	Stable Pen Contact	23
3.4.1	Overheating	25
3.5	Motion Planning and Control	27
3.5.1	Probabilistic Planners	28
3.5.2	Pre-Recorded Motions	31
3.5.3	Resolved-Rate Motion Control	31
3.6	Trajectory Generation	34
3.7	Supervising Algorithm	35
3.8	System Architecture	36
4	EXPERIMENTS	39
4.1	Experimental Setup	39
4.1.1	TIAGo	39
4.1.2	The Laboratory	41
4.2	Comparison of Depth Calibration Methods	41
4.2.1	Plane-fitting Method	43
4.2.2	Extrusion Method	43
4.2.3	Qualitative Comparison	46
4.2.4	Comparison and Conclusion	49

4.3	Qualitative Evaluation of the Depth Correction Module	49
4.4	Qualitative Comparison of Board Detection Methods	52
4.5	Quantifying Pose Estimation Accuracy	54
4.6	Evaluation of Probabilistic Motion Planners	62
4.7	Comparison of Resolved-Rate Motion Controllers	64
4.7.1	Formulation of Metrics	65
4.7.2	Experiments and Results	65
4.7.3	Discussion	69
4.8	Measurement of Arm Backlash	69
4.9	Operating Algorithm Reliability	72
5	CONCLUSIONS	75
5.1	Future Work	76
	REFERENCES	77
	APPENDICES	
A	APPENDIX	81
A.1	Technical Drawings for the Gripper Attachment	81
A.2	Additional Data from the Quantifying Pose Estimation Accuracy Experiment	81

LIST OF TABLES

TABLES

Table 4.1 Pose estimate angular differences between the YOLO submodule and the ArUco submodule.	59
Table 4.2 Pose estimate angular differences between the ArUco submodule and the YOLO submodule. "Single" describes the case where the ArUco submodule can see only one marker, while "double" describes the case where the submodule can see both markers.	61
Table 4.3 Success rate of RRT-Connect at planning the arm-unfurling motion at different distances.	63
Table 4.4 Success rate of RRT-Connect at planning the arm-unfurling and drawing motion at different distances.	64
Table 4.5 Calculated accuracy scores and runtimes for various resolved-rate motion controller configurations.	66
Table 4.6 Length measurements of the backlash deflection marks. Lengthwise deflection calculated via Pythagoras' theorem.	71
Table A.1 Pose estimate angular differences between the YOLO submodule, the ArUco submodule, and the large ArUco submodule at differing distances.	81

LIST OF FIGURES

FIGURES

Figure 3.1	An overview of the various subcomponents making up the system. Yellow indicates swappable submodules and red indicates system interfaces.	9
Figure 3.2	Block diagram showing the signal flow in the Board Detection module. The end results are stored in the Pose Estimate and Mask Estimate slots on the right.	10
Figure 3.3	An image containing a whiteboard forming the training dataset. .	13
Figure 3.4	An image with the whiteboard labeled used as training data. The red polygon is the label.	14
Figure 3.5	Evaluation results of the trained YOLOv11-seg model on the validation set. An incorrect masking can be seen on the picture in the bottom center.	15
Figure 3.6	ArUco marker attached to one corner of the whiteboard	16
Figure 3.7	Depth camera (white) measurements and LiDAR (red) measurements of the whiteboard from approximately 5m of distance. Had the measurements been distortion-free, the white and red lines would be parallel.	17

Figure 3.8	A diagram illustrating the clipped linear interpolation of the calibration coefficient for a single depth measurement. The correction process is a mapping from the uncorrected depth value (pictured as the top axis) to the corrected depth value (pictured as the bottom axis). The white circles indicate the calibration values recorded at different distances. The red lines show the resultant interpolated coefficients for different depth values. The line in pink shows an example correction. . .	19
Figure 3.9	A diagram showing the plane-fitting calibration recording process from a top-down view. The white circles are uncorrected depth measurements. The red stars are the laser scan measurements taken as the true values to be calibrated against. The red line is the plane fit to the laser scan data. The red circle, which is the corrected measurement, is used with the corresponding uncorrected measurement (white circle) to compute the discrepancy.	21
Figure 3.10	A diagram showing the extrusion-based calibration recording process from a top-down view. The white circles are uncorrected depth measurements. The red stars are the laser scan measurements taken as the true values to be calibrated against. The star lighter in color is the closest laser measurement to the uncorrected depth sample in x, and its depth value is taken as the correct value for the depth measurement, that is, the blue circle is taken as the corrected position for the depth measurement. If the calibration factor calculated from the depth discrepancy between the white and the blue circle is applied on the white circle, the red circle is obtained as the corrected measurement.	22
Figure 3.11	Gripper Attachment held by TIAGo	24
Figure 3.12	Gripper Attachment in Compression	25
Figure 3.14	Old Gripper Attachment held by TIAGo	25
Figure 3.13	Assembling the Gripper Attachment	26
Figure 3.15	The TIAGo in its drawing-ready arm configuration.	28

Figure 3.16	An illustration of the thresholds for a motion trajectory. The white circles are the waypoints given by the user, the small red dots are the interpolated waypoints. The light-red circles around both types of waypoints indicate the thresholds for the waypoints: Once the end effector enters the threshold region, the algorithm considers that the waypoint has been reached and targets the next waypoint.	32
Figure 3.17	An illustration of the thresholds for a motion trajectory as used by the skipping PID controller. The white circles are the waypoints given by the user, the small red dots are the interpolated waypoints. The light-red circles around both types of waypoints indicate the thresholds for the waypoints: Once the end effector enters the threshold region, the algorithm considers that the waypoint has been reached and targets the next waypoint. The thresholds for the intermediate waypoints are larger, allowing the controller to pass through them easily.	33
Figure 3.18	The author's name written on the whiteboard by the robot	35
Figure 3.19	Simplified flowchart for the operating algorithm. The parentheses indicate which submodule of the board detector is used. Certain checks and miscellaneous actions in the algorithm have been omitted for clarity.	36
Figure 4.1	A frontal view of the TIAGo robot	40
Figure 4.2	TIAGo in the Mechatronics Laboratory, positioned approximately 5m away from the whiteboard.	42
Figure 4.3	The TIAGo robot recording depth calibration using a flat wall. The camera coordinate frame is illustrated on the robot's head.	42
Figure 4.4	LiDAR readout	43
Figure 4.5	The plane fit to the LiDAR readout (red) and the laser scan translated perpendicularly for comparison (blue)	44

Figure 4.6	The resultant depth multiplier image from the plane-fitting approach.	44
Figure 4.7	Left: Uncorrected depth measurement. Right: Same measurement with correction applied.	45
Figure 4.8	The resultant depth multiplier image from the extrusion approach.	46
Figure 4.9	Left: Uncorrected depth measurement. Right: Same measurement with correction applied.	47
Figure 4.10	Depth measurement error for the top center pixel after correction with the extrusion method vs. the plane fitting method.	47
Figure 4.11	Measured flatness of whiteboard after correction with the extrusion method vs. the plane fitting method.	48
Figure 4.12	RViz screenshot showing uncorrected (green) vs corrected (blue) pointcloud of the board at 50cm distance.	50
Figure 4.13	RViz screenshot showing uncorrected (green) vs corrected (blue) pointcloud of the board at 104cm distance.	51
Figure 4.14	RViz screenshot showing uncorrected (green) vs corrected (blue) pointcloud of the board at 232cm distance.	51
Figure 4.15	RViz screenshot showing side view of uncorrected (dark) vs corrected (light) pointcloud of the board at 170cm distance.	52
Figure 4.16	RViz screenshot illustrating board detection using the depth camera and RANSAC. Top: Pointcloud only, Bottom: Pointcloud and bounding box. Note the extra collision volume for the board lip.	53
Figure 4.17	RViz screenshot illustrating board detection using the depth camera and RANSAC. Top: Pointcloud only, Bottom: Pointcloud and bounding box.	55
Figure 4.18	A successful board detection using the YOLO model. Yellow denotes the proposed mask.	56

Figure 4.19	A false positive board detection from the YOLO model. Yellow denotes the proposed mask.	57
Figure 4.20	RViz screenshot illustrating poor board detection result when only one ArUco marker is in view.	57
Figure 4.21	RViz screenshot illustrating good board detection result when both ArUco markers are in view.	58
Figure 4.22	Whiteboard with the large ArUco marker attached to the middle.	59
Figure 4.23	The TIAGo robot positioned so that both markers are in view. . .	60
Figure 4.24	The TIAGo robot positioned so that one marker is in view. . . .	60
Figure 4.25	Board pose estimate angular differences of the ArUco submodules in the single marker and double marker cases in comparison to the YOLO submodule.	62
Figure 4.26	PID controller trajectory plotted against commanded waypoints.	67
Figure 4.27	SkippingPID controller trajectory plotted against commanded waypoints.	68
Figure 4.28	The TIAGo with its marker touching the whiteboard. While the robot is in this pose, we apply forces to the end-effector by hand to measure deflections.	70
Figure 4.29	The gear transmission in the fifth axis of the robot arm. The internal gear visible in the picture meshes with the external gear painted black.	71
Figure 4.30	The gear transmission in the sixth axis of the robot arm.	72
Figure 4.31	The mark created by the deflections from the fifth axis.	73
Figure 4.32	The mark created by the deflections from the sixth axis.	73
Figure 4.33	Square drawn twice clockwise on the left image, and drawn twice counterclockwise on the right.	74

Figure A.1	Sleeve Technical Drawing	82
Figure A.2	Chuck Technical Drawing	83
Figure A.3	Gripper Attachment Assembly Technical Drawing	84
Figure A.4	Board pose estimate angular differences of the YOLO and ArUco submodules in comparison to the large ArUco submodule.	85
Figure A.5	Board pose estimate angular differences of the ArUco submod- ules in the single marker and double marker cases in comparison to the large ArUco submodule.	85

LIST OF ABBREVIATIONS

2D	2 Dimensional
3D	3 Dimensional
API	Application Programming Interface
DoF	Degrees of Freedom
GAN	Generative Adversarial Network
LiDAR	Light Detection and Ranging
PID	Proportional-Integral-Derivative
PnP	Perspective-n-Point
RANSAC	Random Sample Consensus
REPL	Read-Eval-Print Loop
RGB	Red-Green-Blue
RGB-D	Red-Green-Blue-Depth
RRMC	Resolved-Rate Motion Control
RRT	Rapidly-exploring Random Trees
ROS	Robot Operating System
SLAM	Simultaneous Localization and Mapping
OMPL	Open Motion Planning Library
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 Motivation and Problem Definition

Whatever you now find weird, ugly,
uncomfortable and nasty about a new
medium will surely become its
signature.

Brian Eno

Humans have used drawing as a communication aid for centuries. People draw and write to illustrate and explain concepts, to take notes, to think, and to entertain. The whiteboard, replacing the blackboard, has been the longstanding medium of choice of teachers for their lectures, of teams for brainstorming sessions. With the advent of computers, there has been a gradual digitalization of drawing tools and implements, substituting sheets of paper with screens and pencils with keyboards or stylus pens. The tactile experience of drawing sketches and discussing in front of a whiteboard with a colleague is now in competition with the alternative of looking at websites in a shared video chat.

In this thesis, we aim to create the infrastructure necessary to replicate such whiteboard interactions with a robot as a participant. Our goal is to create a software system that enables our mobile manipulator robot to utilize this communication medium. More concretely, we want to give our robot the capability to detect whiteboards present in the room, and to write and draw arbitrary data on these whiteboards using its arm to hold a board marker. We assume that the whiteboards are flat. The robot is a mobile manipulator with a differential-drive wheeled base and a 7-DoF

arm. This thesis concerns itself with the technical details of implementing such a system. The next section contains an overview of the subproblems involved and the methods proposed to solve them.

1.2 Proposed Methods and Models

In designing robot software to draw on whiteboards, we divide the problem into subproblems. These include locating the whiteboard in the room, estimating its pose and state, moving toward the board using a suitable path, writing on the board using the robot arm, and deciding on what to draw. We opt to focus on well-established, conventional algorithms in implementing the drawing infrastructure for the robot. For the board detection problem, we compare different methods ranging from doing plane fitting on point cloud data to training custom YOLO models to segment whiteboards. We use the path planner built-in to the robot via ROS to chart the robot's movement toward the board. Once the robot has arrived at the board, we solve the problem of motion planning and control to draw on the board with probabilistic planners like RRTConnect, motion control algorithms such as RRMC, and ad-hoc variations thereof. Details about the individual subproblems and methods can be found in Chapter 3 and performance comparisons in Chapter 4.

1.3 Contributions and Novelties

The aim of this thesis is to create a holistic system that can detect, move to, and draw on whiteboards. The bulk of the work consists of partitioning the problem into debuggable sub-components, implementing solutions for these problems based on existing literature and integrating the components together to form a cohesive whole. As also suggested by Bajracharya et al. [1] in their real-life mobile manipulation work, most robotics research studies common robotics subproblems in isolation, attempting to improve their performance with regard to subproblem-specific metrics that have developed over the course of years of research. However, these metrics may not necessarily capture the performance characteristics observed when solving the subproblem as part of a larger, integrated system. The example given by Bajracharya et al. is

that of motion planning. One avenue focused on in planning research is solution optimality and compute cost, but in real life applications of motion planning, success rate/completeness is far more crucial.

This highlights the importance of continued efforts at integration-heavy robotics work. By attempting to utilize the solutions proposed by research fields to their increasingly specialized subproblems as part of a cohesive whole, one can reveal any critical issues that are in the way of adoption of the method in real-life use. As such, robotics research that focuses on real-life testing can act as a guidepost for other fields that focus on the individual challenges involved. This thesis attempts to map the terrain of the difficulties that exist in the mobile manipulation landscape today using whiteboard drawing as a *drosophila* (as used by John McCarthy with regard to AI research). The main contributions of the thesis are in segmenting this example problem into sub-components that map well to the sub-problems studied by the robotics community, and in evaluating some popular solutions to these sub-problems in the context of a real-life problem. Along the way, we develop relatively novel modifications to the algorithms we use to adapt them to our current context. Examples include the multiple-threshold "skipping" trajectory tracking resolved-rate motion controller described in Subsection 3.5.3 and the modified depth camera calibration algorithms we describe in Section 3.3.

1.4 The Outline of the Thesis

This thesis is structured as follows: First, we give an overview of other works tackling the problem of mobile manipulation, whiteboard drawing and creative/collaborative outputs by robots in Chapter 2. Then we provide an overview of the problem we wish to solve in terms of different levels of competencies in Section 3.1. Here we also describe the way we compartmentalize the different aspects of the problem. In the following sections, we study the subproblems themselves: Section 3.3 outlines the depth camera calibration approaches used by others, and explains the method we opted to use for our robot. Section 3.4 highlights the problems faced in ensuring that the pen contacts the whiteboard reliably while drawing, listing the various possible solutions and their trade-offs. In Section 3.5 we describe the motion planning and

control stack used to draw on the board, and the modular formulation used. The trajectory generator that creates the drawing paths is shown in Section 3.6. An overview of the software architecture used to implement the whole package, and a discussion of the advantages the class-based representations and the workflow brings in the development process is given in Section 3.8. Results obtained from real-life executions of the various parts of the system are given and discussed in Chapter 4. Finally, a conclusion and an overview of lessons learned, along with suggestions for future work can be found under Chapter 5.

CHAPTER 2

LITERATURE REVIEW

Mobile manipulation has been an important subfield of research and application for robotics as many everyday tasks can be mapped to a solution that involves a manipulator that can move around. Chebab et al. [2] present a list of various mobile manipulators that have been developed in the past, with a focus on collaboration. Bajracharya et al. [1] develop a holistic mobile manipulation system to locate and grab various items from an "in-the-wild" supermarket in order to evaluate the state-of-the-art in mobile manipulation techniques and how they fare in real-life scenarios in accordance to performance metrics.

Robots that draw on surfaces have a long lineage, going back to entirely-mechanical programmable automata puppets that draw specified trajectories on paper. Plotters are a widespread example of robot use for drawing: they are occasionally used in lieu of printers in business contexts, and a community has spawned around using plotters with generative art techniques to create visual artwork. Plotters can use robot arms instead of a gantry mechanism. Song et al. [3] develop a robot arm plotter that uses pens to draw, focusing on the path and trajectory-generation process to render arbitrary images as curves on paper. A method for drawing paths onto arbitrarily-shaped surfaces using an impedance-controlled robot arm is explored in Song et al. [4]. Haddadi [5] and Weingartshofer et al. [6] also demonstrate drawing on non-flat objects with robot arms. A teleoperation setup for artists to draw using robot manipulators is detailed in Chen et al. [7]. Berio et al. [8] use a humanoid Baxter robot to develop a graffiti drawing system that takes advantage of the robot's overactuated, compliant arms.

Song et al. [9] use diffusion models and vectorization techniques to convert real-life

photos into sketch-like paths for a robot arm to draw on paper. Wang et al. [10] use a style-transfer and GAN based approach to achieve a similar objective. Abrini et al. [11] study the interaction between humans and drawing robots. However, these works do not use humanoid robots.

There is existing precedent for robots that draw on whiteboards. Various hardware configurations have been used for this problem. A relatively early work using mobile manipulators is Waarsing et al. [12]. A popular approach is to attach the robot to the board and have the robot move on it. One configuration uses magnets latch onto the whiteboard and move on the board using wheels. Alternatively, gravity can be used to attach the robot to the board. An example of this configuration being used for paper can be found in Lin et al. [13]. This work is also notable for focusing on using the robot as a creative aide by having the robot generate and draw new additions to user-drawn sketches, using RNNs to enable collaborative ideation. Another work that studies human-robot interaction within a drawing context is Gomez Cubero et al. [14]. Kotani and Tellex [15] automatically generate appropriate paths for the robot to follow to draw an image on the whiteboard. We note that drawing robots that can automatically detect and move toward their drawing surfaces are a relatively underexplored area within the existing body of work, which is one of the goals of our project.

CHAPTER 3

SUBPROBLEMS IN MOBILE MANIPULATION

In this chapter, the various subproblems encountered and the methods used to realize the board drawing system as described in Chapter 1 will be detailed. Comparisons of the proposed solutions to these subproblems according to performance metrics will be given in Chapter 4. The first section of this chapter contains an overview of the problem and how it was split into multiple milestones, and how the project was decomposed into subcomponents. The remaining sections detail previous approaches and the methodology we used to solve the problem. The final section outlines how the system was implemented in software.

3.1 System Overview

As explained in Chapter 1, we aim to create a software system for the TIAGo robot that allows it to move to whiteboards it detects in a room, and draw on them according to behavior specifications. This functionality can be implemented in many ways. Indeed, the different schools of design in the robotics field today would face significantly different problems when attempting to implement this system using their standard approaches. Gervet et al. [16] provide an interesting comparison of some of the existing design methodologies as applied to open-vocabulary navigation.

In our work, we choose to go with the "classical" approach as opposed to an end-to-end or machine learning-heavy solution. We tried to formulate our solution as a collection of modular subsystems, and while some of these subsystems could involve machine learning models, we opted for non-statistical models and algorithms where possible. This decision was motivated by multiple reasons: We wanted the system

to run entirely on-board the robot, save for visualization and interaction tools, which ruled out the larger statistical models that are gaining popularity today. Minimizing the use of learning-based approaches also promised an easier time debugging and developing the system, due to increased interpretability of program behavior, and faster iteration capabilities as fixing problems does not often depend on retraining a model with a new set of data. The problem also seemed to lend itself well to a decomposition into submodules implemented via classical means.

We divide the system into multiple sub-components, each representing different "competencies" expected of the design. This allows us to implement the sub-components corresponding to base behaviors first and gradually layer new modules on top of the system to extend the abilities of the robot, akin to the development process for new behaviors in systems using the Subsumption architecture [17]. The similarities with the Subsumption architecture end there, however, as we do not adopt any of the runtime and formulation constraints that characterise the design system. Our components do not necessarily run concurrently. A single component can run on its own to guide the robot to complete a task, and then hand the control over to two other components running in tandem.

We define the levels of competencies for our robotic drawing system as follows:

1. The robot can draw a specified shape on a board in front of it starting from its home configuration.
2. Starting from a random location, the robot can locate the whiteboard in the room, move to it and draw on it.

For the first competency, we use the Board Pose Estimation module to estimate the board plane from the depth image, and our Motion Planning module to plan and execute the drawing motions with the robot arm. The drawing motions themselves are generated by the Trajectory Generation module.

The second competency involves locating and moving to the board and then handing off control to the set of systems used for the first competency. In order to detect the board, we rotate the robot to search for whiteboards in the room using our Board

Detection module. We estimate the board pose with our Board Pose Estimation module, and move toward the board while avoiding collisions using the pre-existing ROS navigation stack.

A block diagram illustrating the subcomponents can be found in Figure 3.1.

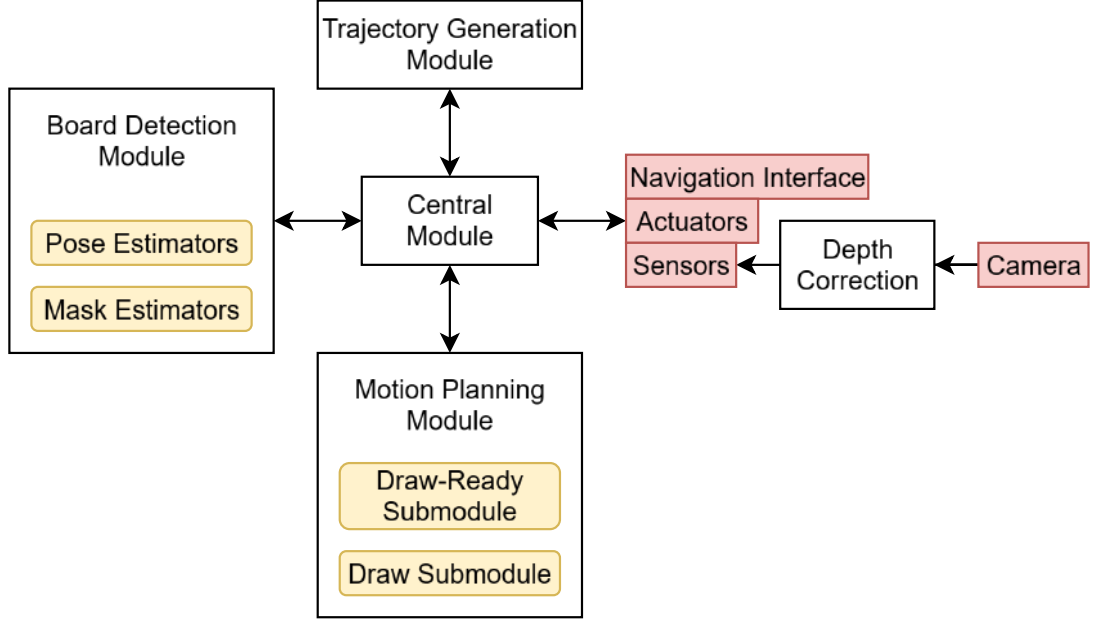


Figure 3.1: An overview of the various subcomponents making up the system. Yellow indicates swappable submodules and red indicates system interfaces.

3.2 Board Detection

Detection and pose estimation of objects is a common challenge in manipulation and autonomous driving research. Our Board Detection module is concerned with detecting whiteboards and estimating the pose of the whiteboard in 3D space with respect to the robot. Due to the wealth of different sensor modalities available on the robot, there are various possible approaches to estimate the whiteboard's pose. Our system uses the board detection module for multiple purposes. In fulfilling the "Draw on a board in front of you" competency 1, the module is used to obtain an accurate estimate of the board pose so that the trajectory generated in the motion planning stage is parallel to the board at a proper distance. For competency 2, the board detector is used in locating the whiteboard in the room before moving toward it.

The board detection module is comprised of multiple board pose estimation submodules. At runtime, the board detection module runs all enabled submodules to get their board pose estimates along with a confidence score for the estimate. The board pose estimates coming from different sensor modalities can then be fused accordingly. The board detector instance used in the work subclasses the module to implement a discrete, modal behavior: The module takes in a mode switch, which determines which submodule should be used. It then uses the instantaneous results from this submodule as the board pose estimate, provided it's above a confidence threshold. The module is structured in a way that it is easy to extend the estimation to feature memory, i.e. temporal weighted averaging, or do sensor fusion to combine estimates from different approaches, but the current modal implementation has been an expedient solution, with enough accuracy to allow the robot to perform its primary tasks. A diagram of the module can be seen in Figure 3.2.

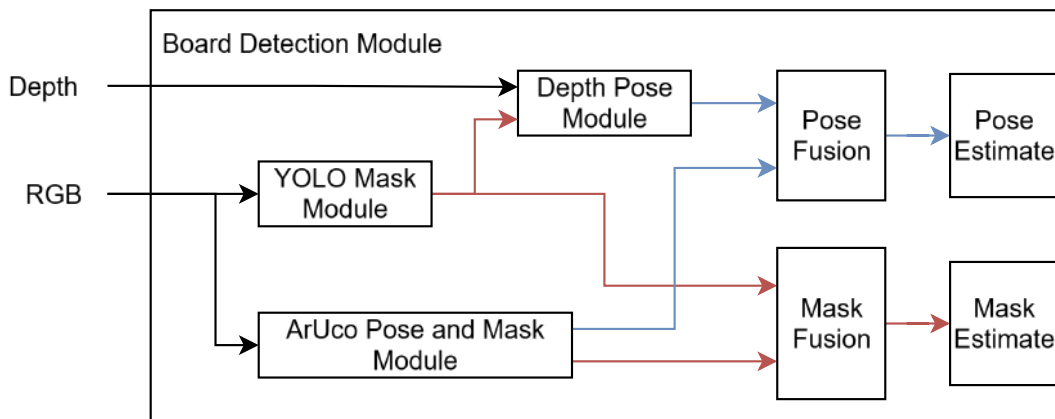


Figure 3.2: Block diagram showing the signal flow in the Board Detection module. The end results are stored in the Pose Estimate and Mask Estimate slots on the right.

The submodules themselves are split into two classes: Mask Estimators, which are concerned with detecting whether a whiteboard exists in the current sensor readout, and providing a mask that segments and specifies where the board is within the sensor readout, and Pose Estimators, which use sensor readouts and optionally the masks generated by Mask Estimators to determine the pose of the whiteboard relative to the world reference frame. The main output of the board detection module is these estimated poses. The masks were included as an unused output to allow for potential extension of the module to supply additional information about the state of the white-

board, such as images and information about what’s written on the board and which parts of it are empty.

The board detection module is also responsible for creating a collision volume for the board to prevent the arm from colliding with the board during the drawing stage.

Plots and illustrations showing the board detector in action can be found in Section 4.4. We present a comparison of the board pose estimation methods outlined below in Section 4.5.

3.2.1 Depth-based Board Pose Estimation

The depth-based board pose estimation approach utilizes the depth image obtained from the RGB-D camera located on the robot’s head. The TIAGo robot ships with software infrastructure to handle the conversion of the dense 2D arrays representing depth images, where each element represents the depth readout for the corresponding element in the RGB image, into 3D point clouds, where each point represents a depth measurement with respect to the robot’s reference frame. This conversion process also includes a configurable subsampling step to reduce the number of points in the point cloud.

We have implemented various methods to estimate the board plane from the obtained point cloud. One such method uses a least-squares formulation to find the plane that minimizes the mean-squared distance of the points to the plane. Since this method tries to minimize the mean-squared error of all points, the presence of points that do not originate from the whiteboard can result in sub-par estimates of the board plane, as the optimizer will try to minimize the distance to points from the foreign object as well as the whiteboard.

We picked the Random Sample Consensus (RANSAC) algorithm [18] as an alternative estimation method that can overcome this problem. This algorithm uses a randomly-sampled portion of the points to estimate the plane, and then scores the suitability of the estimate by checking it against the entire point-cloud, tallying the number of points whose distance to the plane is below a certain threshold (termed "inliers"). This process is repeated for new samplings of points and the highest-scoring

plane estimate is returned as the answer. It's possible to sample any number of points to estimate the plane for a single iteration.

Our implementation of the algorithm samples 3 points per iteration, which permits a unique solution to the plane equation without the need for optimization algorithms. We repeat for 100 iterations and use a distance threshold of 5mm to determine inliers.

We note that our depth-based board pose estimation submodule presumes that the whiteboard comprises the majority of the masked field-of-view of the RGB-D camera. We use this module without any masking to estimate the board pose when the robot is drawing on the board. The robot will be facing the board at a close distance when drawing, satisfying the requisite condition for our pose estimator to work properly without any need of filtering over the point cloud to extract points belonging to the whiteboard.

In longer ranges, we use the depth-based board pose estimator along with a mask estimator module, which helps guide the pose estimation procedure. While implementing this feature we have discovered severe distortion in the depth measurements for longer ranges, as evidenced in Figure 3.7. The calibration procedure we followed to fix these distortions is described in Section 3.3.

It is possible to detect objects from pointclouds [19], but due to the data collection effort this requires, we use alternative approaches to mask our pointclouds, as explained in the next section.

3.2.2 RGB-based Mask Estimation

In order to utilize the RGB information we have access to via the robot's camera, we train a YOLOv11-seg [20] model to detect whiteboards, an iteration on the YOLO [21] series of object detection models. This model outputs a mask, a dense 2D array that indicates which pixels of the image correspond to the detected object. In estimating the board's pose, we use the mask to filter our point cloud such that we're left with the points corresponding to the whiteboard only. Then we use the depth pose estimator module described in Subsection 3.2.1 on the extracted points to estimate the board pose. The combination of the RGB Mask Estimator and the Depth Pose

Estimator from the RGB Pose Estimation module.

As training data, we use a dataset comprising of 81 images of different whiteboards in the laboratory, an example of which can be found in Figure 3.3. We also use 49 images without any whiteboards in them, for a total of 130 images. This number was chosen in accordance with the findings of Hermens et al. [22], who suggest using 120 to 150 images for training. We label the images as seen in Figure 3.4 with polygon masks indicating the whiteboard. The trained model displays reasonable performance on the validation set, with minor artifacts on some masks as shown in Figure 3.5.

YOLO is a light enough model that it can be run on-board the robot with acceptable response times at 441ms on average per inference. We enable the tracking persistence setting of the model to improve estimations over time. We use the model's detection confidences as the confidence values for our estimates.

Evaluating early iterations of the model in real life, we discovered that the model has a tendency to give false-positive results, detecting empty white walls as whiteboards, which we fixed by including images of non-whiteboard objects in the dataset. Details can be found in Section 4.4.

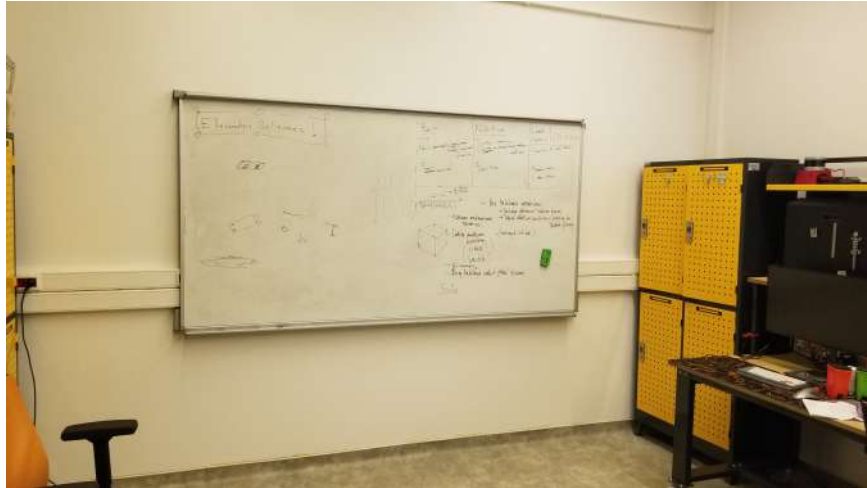


Figure 3.3: An image containing a whiteboard forming the training dataset.

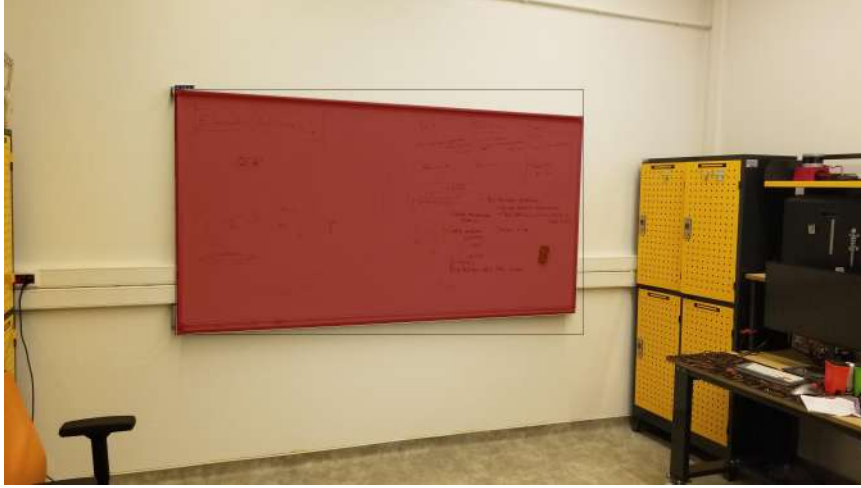


Figure 3.4: An image with the whiteboard labeled used as training data. The red polygon is the label.

3.2.3 Marker-based Board Detection and Pose Estimation

The marker-based approach to board detection and pose estimation relies on ArUco markers that have been placed onto two diagonally-opposing corners of the whiteboard. Taking the size of the ArUco marker as a given, we can use any of the various Perspective-n-Point (PnP) methods to determine the marker’s pose. In our implementation, we use an iterative PnP solver that utilizes Levenberg-Marquardt optimization. If we also know the transform between the marker and the whiteboard, this means that we can estimate the board pose as long as an ArUco marker is in our field-of-view. This means that this module can output masks and board pose estimates simultaneously.

In assigning a confidence value to the estimates, we use the number of markers used in deriving the result. We note slight degradations in performance when only a single marker is detected. This is expanded on in Section 4.4. When the accuracy of the board pose estimate is crucial, e.g. when determining how to move to the board, we wait for the confidence estimate to improve and reject single-marker estimates.



Figure 3.5: Evaluation results of the trained YOLOv11-seg model on the validation set. An incorrect masking can be seen on the picture in the bottom center.

3.3 Depth Camera Calibration

The TIAGo robot has an Orbbec Astra S RGB-D camera mounted on its head. Various parts of the system depend on readouts from this sensor: Board detection depends on the RGB and depth readouts, which in turn affects the downstream tasks of servoing to the board and drawing on it. Thus the validity and accuracy of the sensor data directly affects the reliability of the system.

While testing our software on the robot we determined that the camera made erroneous depth measurements. This was spurred by the observation that the robot could write accurately in certain regions of its field of view but either failed to make contact or collided with the board in other regions. The visualization in Figure 3.7 revealed that the measurements from the laser rangefinder mounted at the base of the robot diverged heavily from the depth measurements obtained from the RGB-D camera on the head.

The distortion observed changes depending on distance. The distortion for short-range measurements appears in a much more regular pattern, as if the camera had been

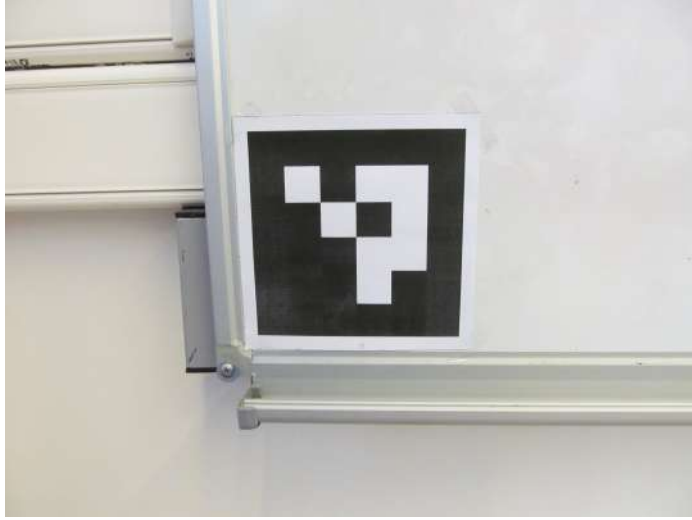


Figure 3.6: ArUco marker attached to one corner of the whiteboard

rotated. This led to an initial diagnosis that the mismasurements were being caused by an error in the head pan motor joint offset, and that the distortions seen at larger distances were a separate phenomenon unrelated to this offset. Attempted corrections to the head pan motor offset ameliorated the situation slightly, but occasional issues persisted.

A scan of the relevant literature yielded various approaches to calibrating structured-light depth cameras. These works used different distortion models to formulate the correction of errors in measurement, which can be broadly separated into the categories of parametric and dense. The parametric models contain a small amount of variables to be determined. Zhang and Zhang [23] use a model comprising of 3 parameters. An et al. [24] model the projector and the camera with pinhole models, inverting the model for the projector, and do a multi-stage calibration to find the intrinsic and extrinsic matrices for these models. Kao et al. [25] build a controlled calibration setup and use a model comprised of 9 parameters.

Conversely, dense models usually feature look-up tables containing correction data for a pixel or a group of pixels. Teichman et al. [26] apply SLAM algorithms on recorded data and use the information to obtain a 2D look-up table called the depth multiplier image, containing multiplicative correction factors for each pixel. Avetisyan et al. [27] use a 3D look-up table, which they fill in according to mea-

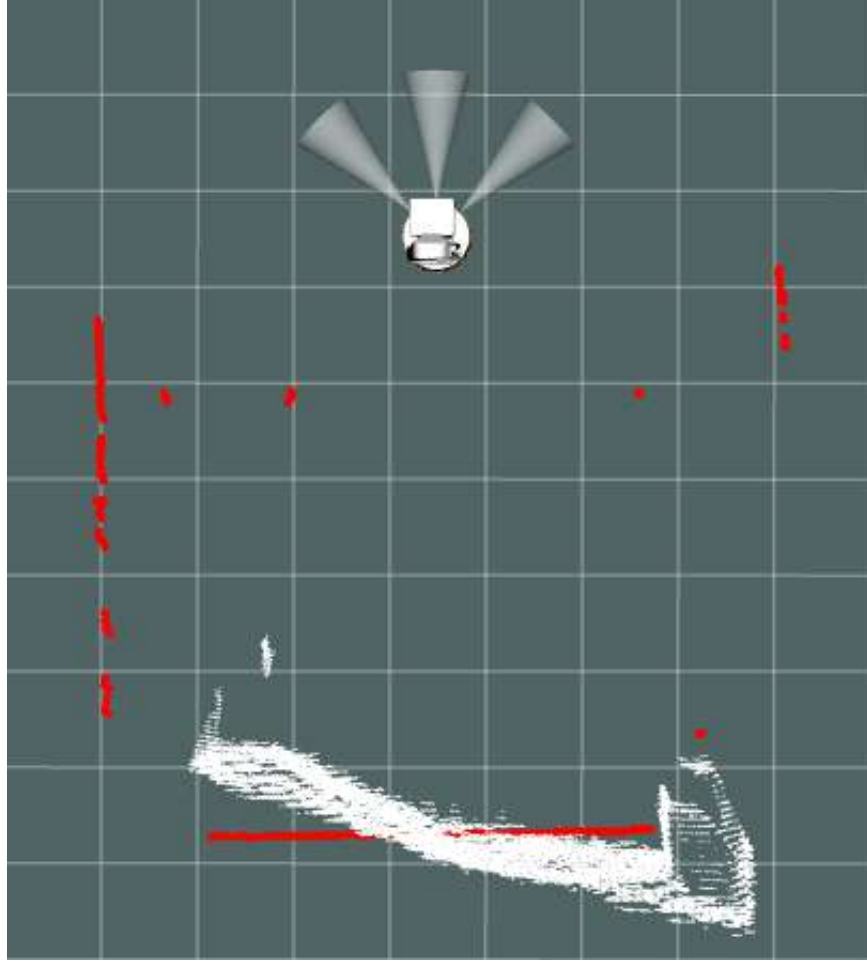


Figure 3.7: Depth camera (white) measurements and LiDAR (red) measurements of the whiteboard from approximately 5m of distance. Had the measurements been distortion-free, the white and red lines would be parallel.

measurements they get of a checkerboard they move around in space, the ground truth position for which they get from another tracking mechanism.

3.3.1 Distortion Model and Correction Algorithm

The camera we want to calibrate is mounted inside the head of the robot. Instead of unmounting the camera from the robot for calibration, which is impractical, we formulate our calibration process to take advantage of the additional sources of information provided by the robot platform. Namely, we use readouts from the laser rangefinder mounted at the base of the robot as ground-truth information to calibrate

the camera against. We can obtain the transformation between the camera and the rangefinder via ROS.

For our distortion model, we modify the depth multiplier image approach used in [26]. We use multiple depth multiplier images recorded at different distances and linearly interpolate between them to find the appropriate multiplier per pixel at runtime. Additionally, we do not do any binning since the differences in distortion in adjacent pixels can be significant, and the computational cost we incur from this is acceptable for our application. Stated more formally, for a point (u, v, d) in the uncalibrated image, we define the calibration function C as

$$C(u, v, d) = \left(\frac{\alpha(u, v, d)u}{f_x}, \frac{\alpha(u, v, d)v}{f_y}, \alpha(u, v, d)d \right) = (x, y, z) \quad (3.1)$$

where $\alpha(u, v, d)$ is the multiplicative calibration factor as a function of the readout's value and image-space coordinates, (x, y, z) is the corrected readout and f_x, f_y are focal distances.

The multiplicative calibration factor α is calculated for a given set of (u, v, d) using a bank of N previously recorded depth multiplier images, $\alpha_i(u, v)$ where $i \in [1, N]$. The uncorrected depth values that correspond to the depth multiplier images are stored as $d_i(u, v)$ where $i \in [1, N]$. We assume that the depth multiplier images have been sorted such that $d_{i-1}(u, v) < d_i(u, v)$ where $i \in [2, N]$. Then the calibration factor for each pixel is calculated by linear interpolation. If the uncorrected d value does not fall between any two recorded depth measurements, the closest recorded depth factor is used without interpolation, as illustrated in Figure 3.8.

$$\alpha(u, v, d) = \begin{cases} \alpha_1(u, v) & \text{if } d < d_1(u, v) \\ \frac{d - d_{i-1}(u, v)}{d_i(u, v) - d_{i-1}(u, v)} (\alpha_i(u, v) - \alpha_{i-1}(u, v)) + \alpha_{i-1}(u, v) & \text{if } d_{i-1}(u, v) < d < d_i(u, v) \\ \alpha_N(u, v) & \text{if } d > d_N(u, v) \end{cases} \quad (3.2)$$

We vectorize the rectification operations so as to not incur big speed penalties during runtime. The algorithm used is described in Algorithm 1. Additionally, we reimplement

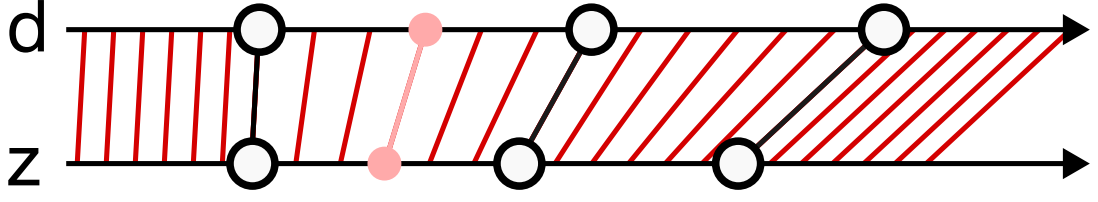


Figure 3.8: A diagram illustrating the clipped linear interpolation of the calibration coefficient for a single depth measurement. The correction process is a mapping from the uncorrected depth value (pictured as the top axis) to the corrected depth value (pictured as the bottom axis). The white circles indicate the calibration values recorded at different distances. The red lines show the resultant interpolated coefficients for different depth values. The line in pink shows an example correction.

ment some ROS functions handling pointclouds, gaining a 100x speedup.

Algorithm 1 Vectorized Rectification Algorithm

```

1: function RECTIFY( $u, v, d, \alpha_{1:N}, d_{1:N}, c_x, c_y, f_x, f_y$ )
2:    $i_1 \leftarrow \operatorname{argmin}_i (|d - d_i|)$ 
3:    $i_2 \leftarrow \operatorname{CLIP}(\operatorname{SIGNUM}(d - d_{i_1}) + i_1, 1, N)$ 
4:    $\alpha \leftarrow \frac{d - d_{i_2}}{d_{i_1} - d_{i_2}} (\alpha_{i_1} - \alpha_{i_2}) + \alpha_{i_2}$ 
5:    $z \leftarrow \alpha \times d$ 
6:    $x \leftarrow \frac{z(u - c_x)}{f_x}$ 
7:    $y \leftarrow \frac{z(v - c_y)}{f_y}$ 
8:   return ( $x, y, z$ )
9: end function

```

The resulting corrected pointclouds are compared to the uncorrected pointclouds in Section 4.3.

3.3.2 Calibration Methods

In order to create the depth multiplier images, two different approaches were tested. Both approaches require the robot to be positioned in front of a flat wall, and make the assumption that the LiDAR and depth camera readout can only sense a flat plane perpendicular to the ground. We only use LiDAR measurements from the front of the robot in a narrow angle (a $2\pi/3$ rad cone), so that other geometry to the sides of

the robot does not affect the calibration. During the calibration process, we exploit the fact that whatever values the LiDAR sees for a given point should stay the same as we move perpendicularly to the scanning plane. In the first approach, we use regression to fit a line to the laser scan of the wall, and use this line to obtain a plane perpendicular to the ground. This effectively gives us the plane equation of the wall. Then we can use this plane to obtain the ground truth depth measurements for every pixel of the uncalibrated depth image. For the plane equation $Ax + By + Cz + D = 0$, we can get the ground truth depth measurements as

$$z = \frac{-D}{\frac{Au}{f_x} + \frac{Bv}{f_y} + C} \quad (3.3)$$

$$x = \frac{z(u - c_x)}{f_x} \quad (3.4)$$

$$y = \frac{z(v - c_y)}{f_y} \quad (3.5)$$

$$\alpha = z/d \quad (3.6)$$

This process is illustrated in Figure 3.9.

In the second approach, we "extrude" the laser scan vertically without fitting a plane. To compute the discrepancy between our raw depth image and the ground truth points, we need to project the depth image from the image plane into world coordinates. However, this operation requires the ground truth depth value, z : $x = z(u - c_x)/f_x$. Since we do not yet know z , we use d instead: $x = d(u - c_x)/f_x$. This substitution holds as long as the robot is facing a flat plane, but degrades to an approximation in other cases. We repeat the same operation for y to obtain the projection of our depth image into world coordinates x, y, d . To compute the discrepancy, for each point from our depth image we find the closest point in x from the laser scan, x_l, y_l, z_l . Then we compute the calibration factor as $\alpha = \frac{z_l}{d}$. A diagram illustrating the process can be found in Figure 3.10 This second approach has the advantage of being more accurate compared to the first approach when the wall has slight deformation and is not perfectly flat.

Charts and illustrations relating to the calibration recording procedure can be found in Section 4.2.

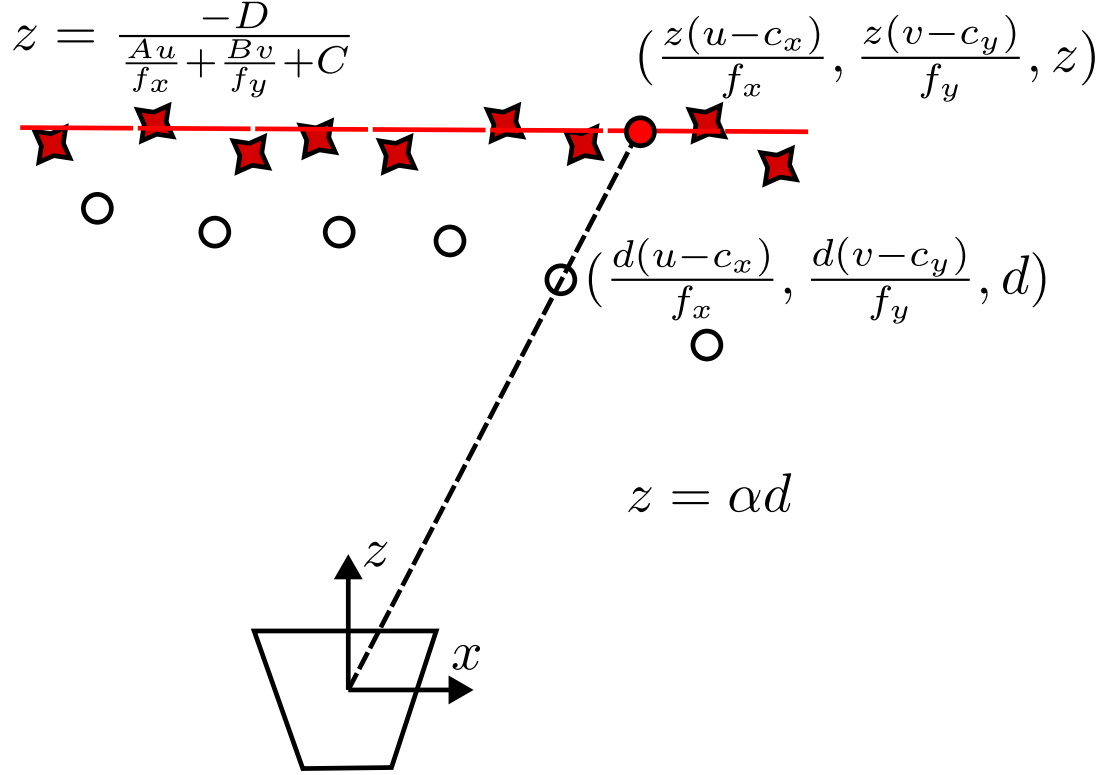


Figure 3.9: A diagram showing the plane-fitting calibration recording process from a top-down view. The white circles are uncorrected depth measurements. The red stars are the laser scan measurements taken as the true values to be calibrated against. The red line is the plane fit to the laser scan data. The red circle, which is the corrected measurement, is used with the corresponding uncorrected measurement (white circle) to compute the discrepancy.

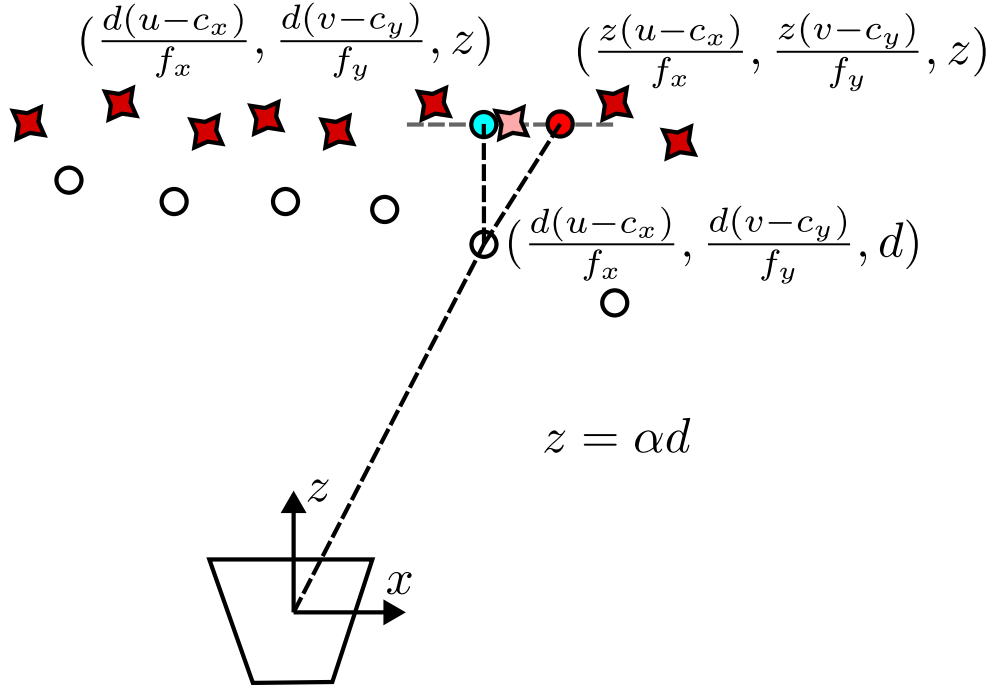


Figure 3.10: A diagram showing the extrusion-based calibration recording process from a top-down view. The white circles are uncorrected depth measurements. The red stars are the laser scan measurements taken as the true values to be calibrated against. The star lighter in color is the closest laser measurement to the uncorrected depth sample in x , and its depth value is taken as the correct value for the depth measurement, that is, the blue circle is taken as the corrected position for the depth measurement. If the calibration factor calculated from the depth discrepancy between the white and the blue circle is applied on the white circle, the red circle is obtained as the corrected measurement.

3.4 Stable Pen Contact

The robot draws on whiteboards using a boardmarker that it holds with its end-effector. When drawing, ensuring that the tip of the pen contacts the whiteboard reliably is critical. A sizable part of the problem is in detecting the board correctly and accurately. The details of the board detection algorithm are discussed in Section 3.2, and the calibration methods used to increase the accuracy of the sensor readings used in this algorithm is detailed in Section 3.3. The other part of the problem lies in determining the pose of the boardmarker held by the robot.

Estimating the pose of a tool gripped by an end-effector is fraught with difficulties. If multiple ways of gripping the tool are possible, one needs to introduce some form of sensory feedback to measure the tool’s relation with respect to the gripper frame. Since the tool will be subjected to forces while it’s being held, various problems can stem from the tool slipping, meaning that the effect of contacts on the tool pose will need to be measured or estimated. Tactile sensors like the one designed by Kuppuswamy et al. [28] are one solution to these issues. An additional problem arises from the fact that part of our tool, the tip of the boardmarker, is deformable. Modelling this deformation accurately presents a challenging problem.

The factors listed above form a compelling case against a detailed mathematical modelling and estimation approach. Therefore we explored other methods of ensuring pen contact with the whiteboard. Usage of a force/torque sensor to detect tool contact is an approach used often, however our robot did not have a force/torque sensor installed on its arm. We prototyped a force/torque sensor using current readings from the motors of TIAGo’s arm, but the current noise and chattering due to the arm controllers meant that attempting to extract reliable and stable force/torque readings from the motor currents would require substantial filtering work, extending outside the purview of this project.

The solution we used instead was to design a mechanically compliant gripper attachment for the end-effector, constraining the boardmarker’s pose and allowing movement in only a single axis parallel with the pen longitudinally. The design can be seen on the robot in Figure 3.11. This allowed us to know the marker’s pose in re-

lation to the gripper at all times. To ensure that the marker made contact with the whiteboard, we command the robot to "push in" the pen while drawing. In Figure 3.12, the attachment can be seen under an exaggerated amount of compression. The high degree of compliance in our gripper design compensates for errors in board pose estimation, pen tip deformations, and other unanticipated conditions. As such, we can convert the force-control problem of drawing on the whiteboard into a position-control problem. The process of inserting the pen into the attachment is illustrated in Figure 3.13. Technical drawings of the gripper are given in Appendix A.1. Similar approach have been pursued by Gomez Cubero et al. [14] and O'Dowd [29], where compliant attachments are used for drawing robots. Notably, in O'Dowd, a sensor is used to measure the pencil deflection to draw more precise strokes in the work.



Figure 3.11: Gripper Attachment held by TIAGo



Figure 3.12: Gripper Attachment in Compression

3.4.1 Overheating



Figure 3.14: Old Gripper Attachment held by TIAGo

A problem we faced while using the first version of the compliant gripper attachment was overheating. Our TIAGo robot has a parallel gripper with two fingers, both of

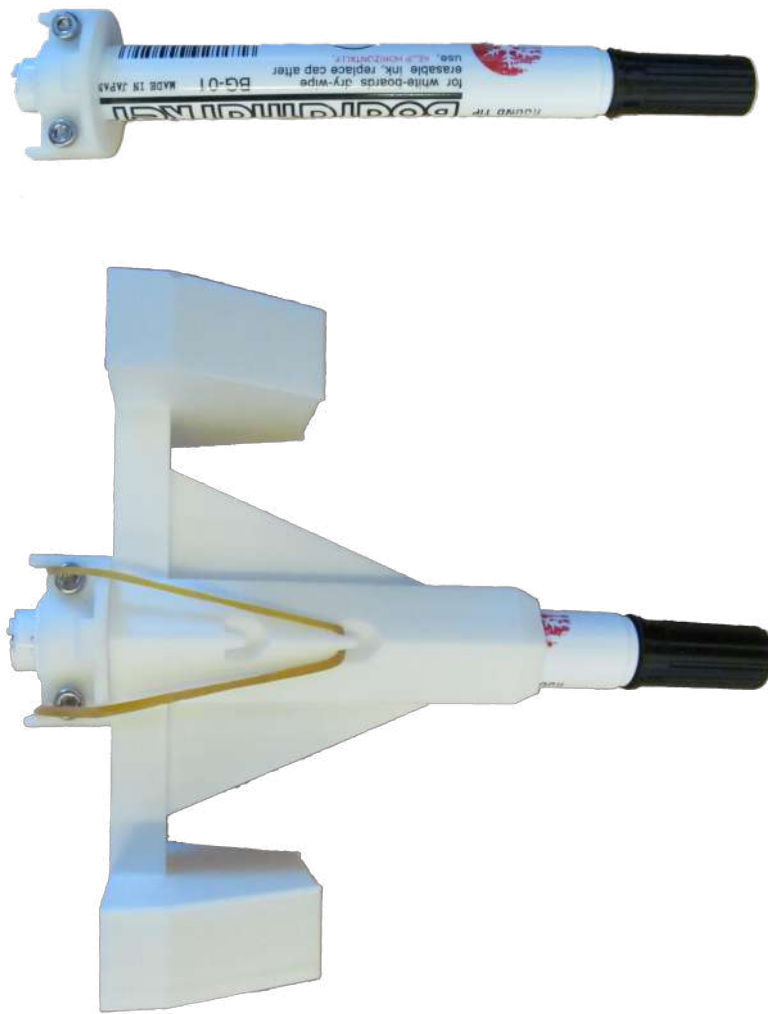


Figure 3.13: Assembling the Gripper Attachment

which have separate Dynamixel motors as actuators. While drawing and while idling, the motors would have to exert a certain amount of force to keep the gripper attachment in place, and also to counteract any forces the marker faced when it contacted the board. The robot can be seen actively gripping the attachment in Figure 3.14. This current draw caused the motors to heat up. The software running on the TIAGo has an overheating watchdog that shuts down any motor whose temperature exceeds $60^{\circ}C$. Once the overheating error flag has been raised for a motor, there is no easy way of clearing the flag, requiring a system restart to bring the motor back into operational condition. This posed a major problem as the motors would heat up after short drawing sessions to the order of 10-20 minutes. The gripper control interface

of the robot allowed us to limit the gripper motor current and was the recommended method to prevent overheating, but the gains in operational lifetime were limited as the motors still reached the temperature threshold after a short amount of time.

We were able to mitigate the problem by making certain changes on the design of our gripper attachment. We changed the width of the attachment so that the fingers would be opened far wider when holding it. With the gripper fingers now close to their mechanical stops, the gripper motors were not the only parts withstanding the forces encountered by the boardmarker. Additionally, the finger slots of the gripper attachment were redesigned to close out the back side of the gripper fingers. This let us put on the gripper attachment and hold it in place without applying any pressure from the gripper, reducing the motor effort substantially. This new iteration of the attachment was used for multiple hours without resulting in motor overheating errors.

3.5 Motion Planning and Control

The motion planning and control module is used to command the TIAGo's robot arm. We separate the motions needed to draw on the board into two actions. The first action is to move the arm from its resting position, in which the arm is tucked in close to the torso, to an extended position where the end-effector is facing toward and close to the board. This position is pictured in Figure 3.15. The second action is to follow the path given by the upstream trajectory generator module with the end-effector in order to draw on the board.

The reason for this separation is that the primary concerns during the two actions are different: For the first action of moving the arm into a drawing-ready configuration, the reliable generation of a collision-free, smooth path is paramount. For the second action, the chance of collision is lower, following the commanded path accurately and without delay is more important.

For the two actions we evaluate the performance of the pre-existing planners in ROS' MoveIt package and supplement some alternatives of our own. We structure the motion planning and control module with slots for the submodules that handle the two actions. One can then take any conforming planning or control submodule and in-

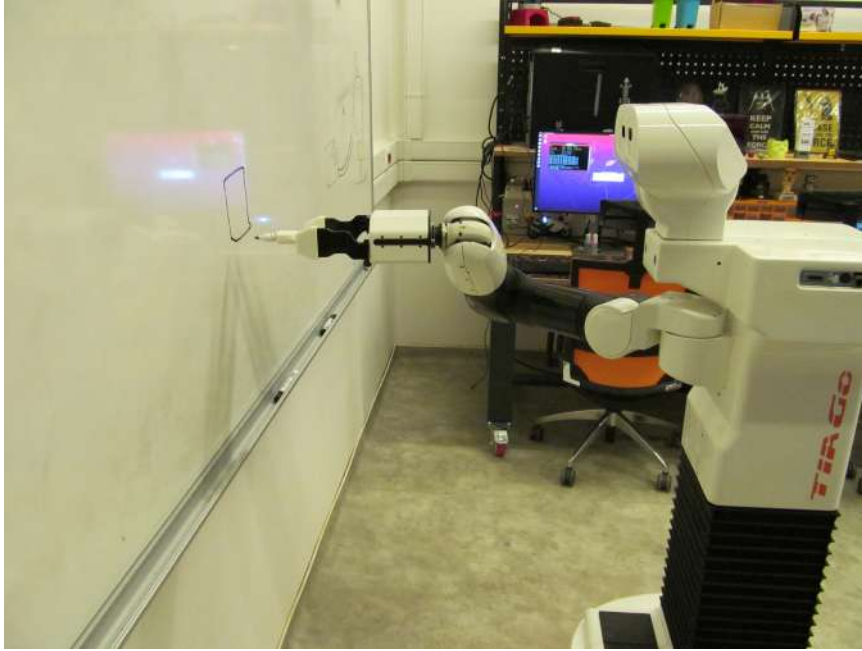


Figure 3.15: The TIAGo in its drawing-ready arm configuration.

sert it into one of the interfaces to create a new motion control stack. This allows us to change the motion planning and control algorithms used by the robot at runtime, and save combinations and configurations of algorithms that demonstrate good performance.

All planning and control submodules descend from a parent `PlanningModule` class. The different subclasses of this module representing the different approaches to motion planning and control are explained in the following subsections.

3.5.1 Probabilistic Planners

One specialization of the planning module allows for interfacing with the different motion planning algorithms that are included with MoveIt. While MoveIt has various backends, the backend with the highest amount of different planners is the OMPL [30] backend. This backend provides access to the motion planning algorithms available in the OMPL project. Most of the planners included in OMPL are probabilistic. This stochasticity means that the plan returned for a given motion task will not necessarily be the same in every run. Indeed, the generated plans show a large amount of variance

in practice.

Probabilistic planners are not guaranteed to find a solution in finite time, and the MoveIt interface allows one to define a plan timeout duration, after which the planner gives up, and a planning attempt setting, which lets the planner try finding more solutions after finding one correct solution, giving the planner a chance to try for shorter trajectories within the allotted time.

We discovered that many of the OMPL planners supplied with MoveIt often failed to produce answers within our specified timeout duration of 2 seconds. Certain planners would produce trajectories that involved the robot’s arm doing many unnecessary movements, or output trajectories the execution of which result in many near-misses with the objects in the environment. These behaviors are undesirable because unpredictable motion trajectories look unnatural and can pose a danger to any humans around the robot. For our project which aspires to have robots collaborate with humans, the robot should move in a natural, predictable way. Additionally, the collision close-calls pose a risk because they leave little margin for error on the board detection side of the system which is responsible for creating the collision volumes for the whiteboard and pen.

The OMPL planners each have different options and can be configured to alter their characteristics. However, MoveIt does not present a convenient interface to tweak these parameters programmatically during runtime. All configuration needs to be done system-wide through configuration files, which is an opaque process. This means that tuning the planner parameters has a high iteration time. Combined with the varying configuration options for the different planners, we opted not to spend time attempting to improve the configurations of the different planners available through MoveIt.

For the first action of moving the arm to a drawing-ready pose, informal evaluations determined that the RRT-Connect [31] implementation in OMPL provided reasonably predictable point-to-point trajectories and found a motion trajectory so long as the robot’s distance to the board was within a specific range. Getting too close to the board can result in unpredictable trajectories that involve maneuvering the arm from the sides of the robot, and going too far can quickly increase the failure rate of

planning.

When going toward the board, we use collision volumes for the board and the pen held by the robot. The collision volume for the board is created and updated automatically by the board detection module. Both collision volumes depend on measurement data regarding their respective sizes entered into the program beforehand.

MoveIt presents a special "cartesian planning" interface for the OMPL backend. Trajectories generated through this interface ensure that the end-effector of the robot moves smoothly through the supplied waypoints, interpolated in cartesian space. This is ideal for drawing, which requires the end-effector to follow the given path and not deviate from the trajectory in-between the waypoints. The details of our trajectory generation module are explained in Section 3.6, but it bears mentioning here that the trajectory generation module also interpolates the waypoints for a given drawing command, populating the path with sub-waypoints so that the end-effector tracks the desired lines.

For the task of drawing on the board, using the point-to-point interface of the MoveIt planners resulted in trajectories that would cause the pen to contact the whiteboard for the supplied waypoints and sub-waypoints, but the path taken in-between often caused spurious lines and squiggles that had no bearing with the commanded drawing trajectory, or the pen would simply break contact with the board resulting in unrecognizable shapes. Furthermore, since the arm operates in very close vicinity of the board during the drawing process, the maneuvers that the planner would attempt while moving from one point on the path to the other would often result in collisions with the board. Individually planning the many trajectories from one waypoint to the other would eventually lead to planning failures, owing to the fact that many separate trajectory plans would need to succeed for us to obtain a working motion plan.

The cartesian planning interface fared better. Provided the planner was able to come up with a motion plan, execution of this plan would result in uniform, straight lines that matched the commanded trajectory. However, obtaining a motion plan from the cartesian planning interface was highly unreliable, and would fail more often than it returned a working plan for our drawing planning problems. Owing to the stochasticity of the planners, the failure could occur while planning different points

of the trajectory, and each failure would require a replan of the entire task.

We present a more quantitative analysis of probabilistic planners in Section 4.6.

3.5.2 Pre-Recorded Motions

The unreliability of the MoveIt planners stemming from their probabilistic nature motivated us to create a module that would execute pre-planned motions that were stored in advance for the first action of moving the arm into the drawing configuration. Since this task was largely static in that there were no extreme changes to the distance to the board between runs, it was feasible to simply generate a satisfactory, natural-looking plan once ahead-of-time and then reuse it for subsequent runs.

This worked acceptably in practice, so long as the robot's distance and orientation with respect to the board did not change dramatically. This precondition was straightforward to satisfy, given that during runs the robot navigates by itself to the board, targeting a pre-specified pose with respect to the board. However, in cases of extreme failures within the board detector module, the target pose would be mis-computed and the robot would stop too close or too far to the board, or orient itself in a skewed manner with respect to the board plane. The pre-recorded approach has no way of compensating for these errors, unlike the probabilistic planners, and this may result in collisions if the program is not stopped in time.

A special case that had to be accounted for was when the robot would receive a drawing command after having finished drawing something else. One version of the pre-recorded motion module contains a check so that the recorded motion does not execute unless the robot arm is in its home configuration.

3.5.3 Resolved-Rate Motion Control

The drawing part of the robot's motion is highly dynamic, the commanded path for the end-effector is likely to change in every run. This precludes the use of our pre-recorded motion modules for this action. As an alternative to the potentially unreliable probabilistic motion planners for the drawing problem, we implement different

reactive motion controllers through the velocity interface of the arm actuators.

We implement a resolved-rate motion controller [32] for the arm that uses a PID control scheme to track the end-effector pose:

$$e(t) = J(q)^{-1}v \quad (3.7)$$

$$\dot{q} = K_P e(t) + K_I \int_0^t e(t)dt + K_D \frac{d}{dt}e(t) \quad (3.8)$$

$$(3.9)$$

In order to follow a given trajectory, we command the controller to move toward a waypoint until the absolute error between the end-effector's pose and the target pose is within a specified threshold. At this point we change the target of the controller to the next waypoint. An illustration of the thresholding is given in Figure 3.16

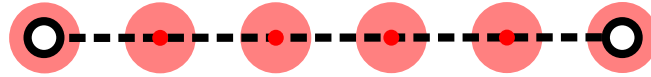


Figure 3.16: An illustration of the thresholds for a motion trajectory. The white circles are the waypoints given by the user, the small red dots are the interpolated waypoints. The light-red circles around both types of waypoints indicate the thresholds for the waypoints: Once the end effector enters the threshold region, the algorithm considers that the waypoint has been reached and targets the next waypoint.

In practice, this approach is considerably more reliable than the probabilistic planners. The quality of the drawing depends on how well-tuned the PID controller is. Ill-tuned controllers can cause oscillatory artifacts and overshoots in the output. In certain cases with high amounts of steady-state error, the controller may fail to bring the end-effector within the threshold of the waypoint, causing hangups.

If the controller fails to bring the end-effector within the threshold boundary of the target in a reasonable amount of time, the movements of the robot look unnatural as it appears to pause at random parts of the drawing. There are various parameters that can be adjusted to mitigate this problem, with their associated tradeoffs. As mentioned earlier, the trajectory generator module interpolates the drawing command it has been

given, adding sub-waypoints to fill in the lines before passing the waypoints to the motion planning and control module. Increasing the amount of sub-waypoints can help the controllers draw straighter, more uniform lines, but it can have the opposite effect if the robot has to pause to exactly arrive at each of the many sub-waypoints that form the line it is drawing. An option is to increase the arrival threshold for the waypoints, but this comes with the concomitant decrease in drawing precision. A square drawn with lenient thresholds will have rounded corners.

We would like the original waypoints specified as part of the drawing to be precise, but the end-effector does not need to show the same care for the interpolated sub-waypoints used to straighten the path between the waypoints. This lead us to design a variation of our PID controller module called the Skipping PID controller. In this variation, instead of a single threshold, a primary and secondary threshold is defined. The strict primary threshold is used for waypoints belonging to the original draw command, while the more lenient secondary threshold is used to check for the sub-waypoints forming the paths between the actual waypoints. A diagram is given in Figure 3.17. This approach helps the controller avoid pausing while drawing straight lines, but still requires the controller to reach the corners of the shapes that it is drawing. With a sufficiently high number of sub-waypoints inserted in the path, this approach is not unlike the sequential composition technique as this process can be viewed as the "controllers" for the various sub-waypoints taking over as the end-effector moves through the path.

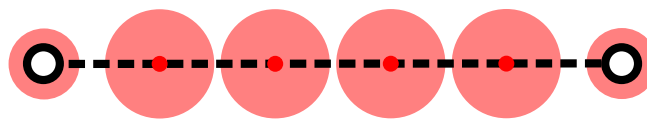


Figure 3.17: An illustration of the thresholds for a motion trajectory as used by the skipping PID controller. The white circles are the waypoints given by the user, the small red dots are the interpolated waypoints. The light-red circles around both types of waypoints indicate the thresholds for the waypoints: Once the end effector enters the threshold region, the algorithm considers that the waypoint has been reached and targets the next waypoint. The thresholds for the intermediate waypoints are larger, allowing the controller to pass through them easily.

In our evaluations, we discover that forgoing the path interpolation and feeding the waypoints directly to the controller with no sub-waypoints can lead to smoother and faster drawing motions compared to other approaches. This comes with the tradeoff that the lines will not be perfectly straight, however.

Our controller does not do any collision checking or attempt to avoid singularities. This means that commanding the arm to move to an unreachable pose will not signal any errors and may result in collisions and erratic movement. In practice this is not a critical problem as we can command the robot to stay at a reasonable distance from the board, which yields a large enough working plane for most intents. Collisions with the board are unlikely to occur since the board, which is the only collider object that we model, is not in the way of the arm’s movements during the drawing process. However, if needed, the controller can be augmented to avoid collisions and singularities. As an example, Haviland and Corke [33] provide a quadratic programming-based formulation of the resolved-rate motion controller that mitigates these problems.

3.6 Trajectory Generation

The trajectory generation module is responsible for converting the drawing commands supplied through the drawing interface into a path that the robot arm can follow. The drawing API takes in a list of 3D points. The first two dimensions of the point correspond to 2D positions on the board, and the last dimension is a boolean indicator that specifies whether the pen is touching the board or not. Through this interface, upstream modules can specify arbitrary disjoint piecewise linear paths for the robot to draw. An example of a disjoint path drawn by the TIAGo is pictured in Figure 3.18.

The module applies a frame transformation to the given points to align them with the board pose as currently detected by the board detection module. If specified, the module also adds sub-waypoints in between the given points by linearly interpolating between the points. These sub-waypoints can be helpful in ensuring the arm follows the trajectory exactly when using some of the submodules of the motion planning and

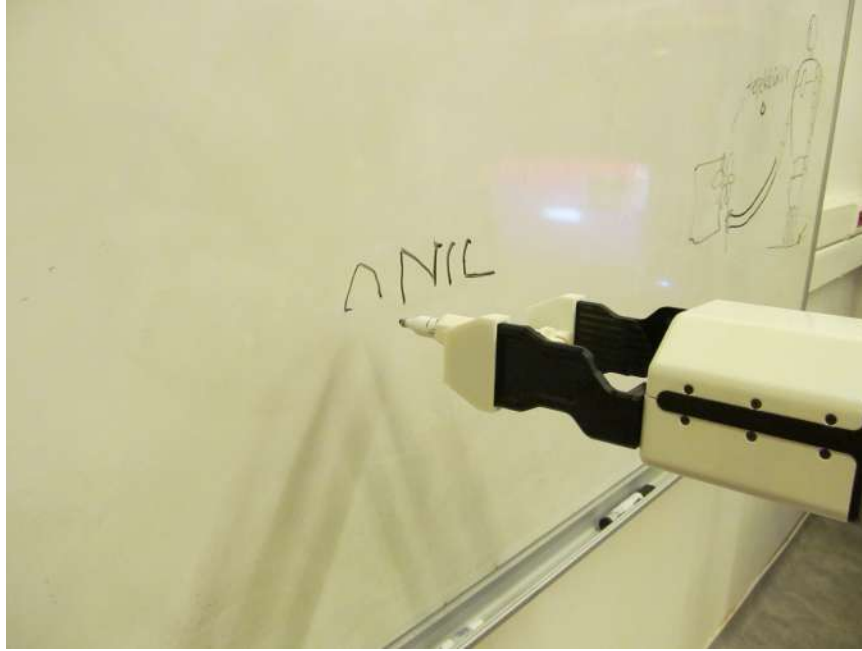


Figure 3.18: The author's name written on the whiteboard by the robot

control module.

This module is also responsible for generating the pose that the robot should move its end-effector to when preparing to draw. In generating this pose and when creating the drawing path, the module uses stored measurements of the pen and the gripper attachment.

3.7 Supervising Algorithm

We use the functionality provided by the modules outlined in the previous chapters to implement different subroutines representing the behaviors that we expect from the system in a central module. This central module has routines for finding boards, moving to boards and stopping within a specified distance, moving the robot's arm to its drawing-ready configuration on the board, drawing arbitrary trajectories and pre-set shapes, and miscellaneous routines for visualizations, interfacing with the robot, debugging and collecting data. To demonstrate the fulfillment of the objectives we listed as the aim of our system, namely, detecting and moving toward a board in the room and then drawing specified trajectories onto it, we devise the operating algo-

rithm illustrated in Figure 3.19. When this routine is run, the robot rotates until it detects the whiteboard using the ArUco markers. Then it rotates to face the center of the board, to ensure that the camera can see a large portion of the board. Next, the YOLO-based board estimator is used to extract the pose of the board, which is used to determine where the robot should move to be positioned approximately 1.5m away from the board. The robot then moves to this spot, and uses the depth information again to generate a more precise pose estimate for the board. This is used to move the robot to its actual drawing distance of approximately 1m. Then the drawing routine is called, which uses the motion planning submodule to plan and execute the unfurling and drawing motions of the arm.

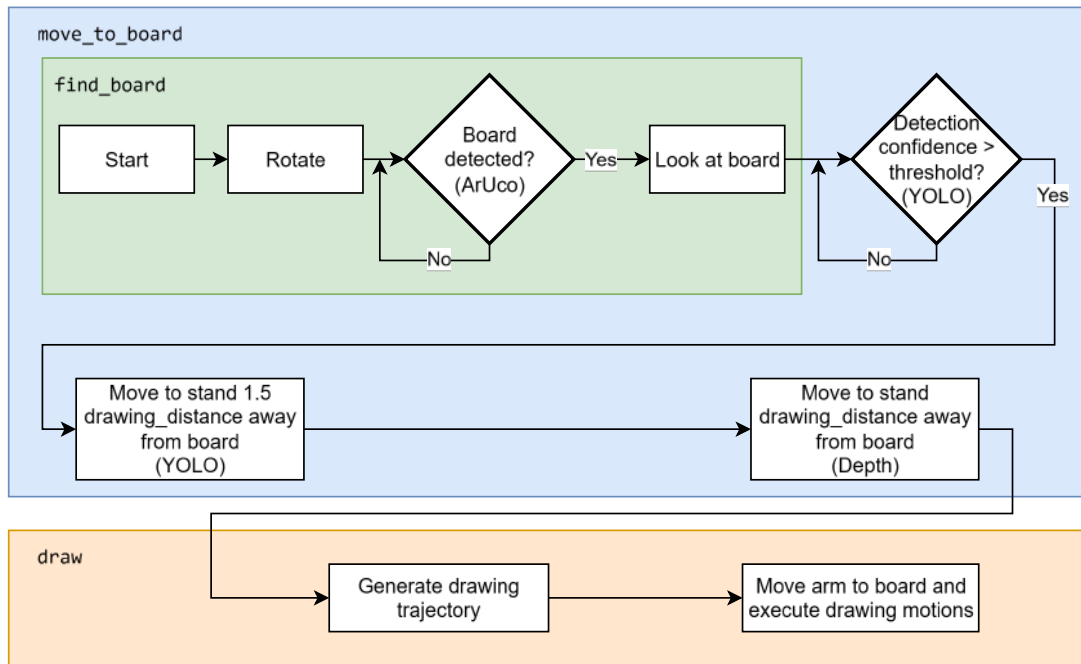


Figure 3.19: Simplified flowchart for the operating algorithm. The parentheses indicate which submodule of the board detector is used. Certain checks and miscellaneous actions in the algorithm have been omitted for clarity.

3.8 System Architecture

This section describes the approach taken in implementing the system in software. The TIAGo robot provides a full ROS environment, with interfaces to its sensors and actuators as ROS topics and services. ROS encourages a node-based development

method, with the nodes for different functions running under different processes concurrently and exchanging data via the pub-sub mechanism. While this runtime model has advantages in terms of performance and uniformity easing extensibility, it has certain characteristics that are disadvantageous when iterating over prototype designs. Implementing different modules as separate ROS nodes means that the nodes will need to be launched separately, or that a launchfile to handle this operation will need to be written. Once launched, the nodes are expected to loop continuously and react to messages received through subscribed topics. This means that nodes will need to be structured and written with a command loop in mind. Transfer of control between nodes is not standard as nodes are usually expected to keep running until the system is shut down. New message types and topics will need to be defined as new data exchange needs arise. Defining new message types requires a recompile and cannot be done during runtime, hurting interactive development prospects.

Due to these reasons, we create an abstraction library that wraps the capabilities of the TIAGo robot's ROS interface and presents them as a Python class. We make this interface available online. This interface allows us to implement all modules in the same process, making transfer of control and data exchange between modules easy. At runtime, we can use tools like IPython for post-mortem debugging to investigate errors after they have occurred and hot-code reloading to update our code while it's running. Running a Jupyter server on the robot lets us render the plots created in the on-board Python process on our remotely connected development computers.

The TIAGo interface subscribes to all sensor and actuator topics and saves incoming updates into buffers. The application programmer can then access the most recent readouts by accessing the relevant slots of the TIAGo class. Any unused sensor and actuator subscriptions can be disabled to save on processing power. To make working with coordinate transforms easy, the interface automatically converts all transforms to and from *spatialmath-python* SE(3) transforms, which are part of a library written by Corke and Haviland [34]. Many additional convenience functions are included to provide commonly-used functionality such as moving to targets, getting transforms, and formatting sensor readouts.

The drawing system is implemented as a subclass of the TIAGo class. High-level

commands are implemented as class methods. This lets us run and debug different system behaviors from the Python REPL separately for interactive testing. As an example, the *draw_square* command calls on the *draw* command and the *move_arm_to_board* command to first move the arm into an appropriate configuration and then draw a square. We can test each function separately to verify their behavior before calling on the higher-level command of *draw_square*.

We use classes extensively while implementing our system. Information about the environment, such as the dimensions of the board, the relations of the ArUco markers to the board and the dimensions of the gripper attachment and boardmarker are stored in separate classes. We use two separate instances of the gripper attachment information class to store the measurements of the two different versions of the gripper attachment. As such, we can update our code with new versions and implementations of the same functionality without throwing our old code away. We can then switch back to our old implementations easily if the need arises.

The implementation of the motion planning and control module is an example of this. Each of the motion submodules is part of a class hierarchy with specializations and mixins overwriting methods to implement their functionality. To give a concrete example, the cartesian motion planner is a subclass of the RRTConnect motion planner with the cartesian planning mixin. The RRTConnect class is a subclass of the OMPL class, which inherits the Probabilistic class, which inherits the base class, PlanningModule. We can store the different combinations and configurations of submodules we find useful as subclasses. Then we can switch between these during runtime at will.

For the depth calibration module, we write a ROS node that takes in the uncorrected depth image and publishes the corrected depth image and pointcloud on new topics. We implement this as a ROS node since it's likely to be individually useful in other endeavours and running the republisher as a separate process is advantageous in terms of speed.

CHAPTER 4

EXPERIMENTS

In this chapter we present the experiments we conducted and the results we obtained. We also offer some discussion regarding the results. Detailed explanations of the submodules under test can be found in Chapter 3.

4.1 Experimental Setup

This section details the objects and environment used in the experiments.

4.1.1 TIAGo

TIAGo is a mobile manipulator robot designed by PAL Robotics. An image of the robot is given under Figure 4.1. While multiple configuration options for the robot exist, the one used in this thesis has a differential-drive wheeled base and a single 7-DoF arm with a parallel gripper as its end-effector. The robot bears some resemblance to humans. Its head is motorized to control its pitch and yaw, and it has an actuator that can lift its torso. In terms of sensors, TIAGo has an RGB-D camera on its head, and a planar laser scanner mounted in its wheeled base. There are three ultrasonic sensors located at the rear of the robot's base.

Its onboard computer ships with a Robot Operating System (ROS) environment pre-configured with topics and services to access the various sensors, actuators and functionalities. We develop and utilize our own Python wrapper library over this ROS interface when interacting with the robot. We detail this interface in Section 3.8.



Figure 4.1: A frontal view of the TIAGo robot

We chose to use this robot to implement our drawing project because its set of sensors and actuators seemed suitable for the task. The mobile base and the depth camera serve to detect the board and position the robot somewhere suitable to draw on it. The overactuated arm allows for multiple configurations for the same end-effector pose, providing us with more options when motion planning for drawing.

In implementing the system, however, we faced certain difficulties regarding the hardware, and were not able to find software solutions addressing the problems at the ready. In particular, we had to correct the depth readings ourselves, had to work around motors overheating since restoring operation once any motor heated up too much was very time consuming, and could not shut down the robot programmatically. Despite these difficulties, the ROS environment that was shipped with the robot proved useful during development.

4.1.2 The Laboratory

The experiments were conducted in the Mechatronics Laboratory, which is a part of ROMER. The laboratory contains a whiteboard on one of its walls which is 1.22m tall and 2.43m wide, hung up such that it is 0.88m above ground. In tests involving the robot detecting and moving to the board, the robot was started from poses approximately 5m away from the whiteboard, as shown in Figure 4.2.

4.2 Comparison of Depth Calibration Methods

This section provides sample results of applying the depth calibration methods described in Section 3.3.2. The calibration was done using the wall pictured in Figure 4.3. Note that for the figures in the following subsection, we use the camera coordinate frame shown in Figure 4.3 with the y axis inverted.



Figure 4.2: TIAGo in the Mechatronics Laboratory, positioned approximately 5m away from the whiteboard.



Figure 4.3: The TIAGo robot recording depth calibration using a flat wall. The camera coordinate frame is illustrated on the robot's head.

4.2.1 Plane-fitting Method

The plane-fitting approach uses the laser scan to estimate the wall as a plane, and then uses this plane equation to calculate the depth multiplier image. The results of the laser scan can be seen in Figure 4.4. In this figure, we note the presence of some measurements not belonging to the wall used for calibration. While these can be filtered out by changing the angle cutoff for the laser scan, this is not necessary in practice. The RANSAC algorithm used to fit a plane to the LiDAR data is tolerant against spurious data, and we can observe that the plane fitting is successful in Figure 4.5. The depth multiplier image shown in Figure 4.6 indicates a regular distortion in the yaw axis. The corners of the image have more extreme amounts of distortion, likely caused by the lens. Applying the multiplier image back on our depth image, we can recreate our ground truth plane again, as shown in Figure 4.7.

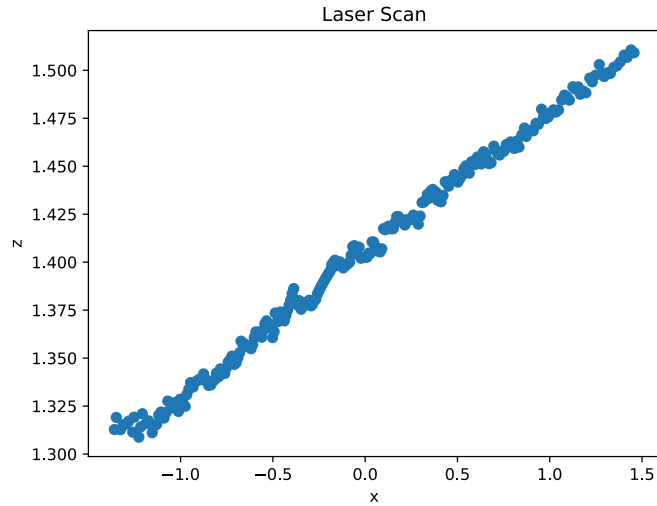


Figure 4.4: LiDAR readout

4.2.2 Extrusion Method

Using the extrusion method we can perform our calibration without using any plane-fitting algorithms. The depth multiplier image we obtained from this method can be found in Figure 4.8. Compared to the image from the plane-fitting method, Figure 4.6, there are more vertical artifacts in this one. A similar phenomenon can be observed

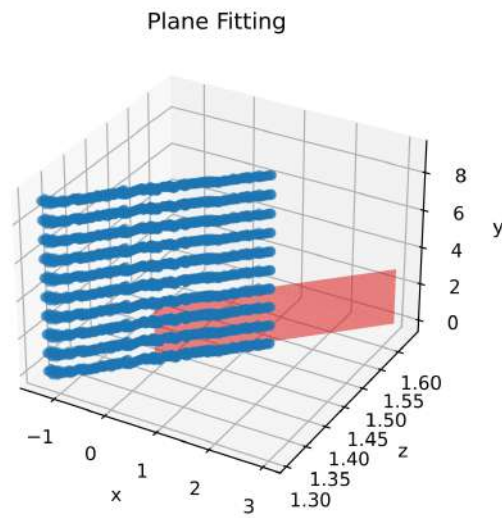


Figure 4.5: The plane fit to the LiDAR readout (red) and the laser scan translated perpendicularly for comparison (blue)

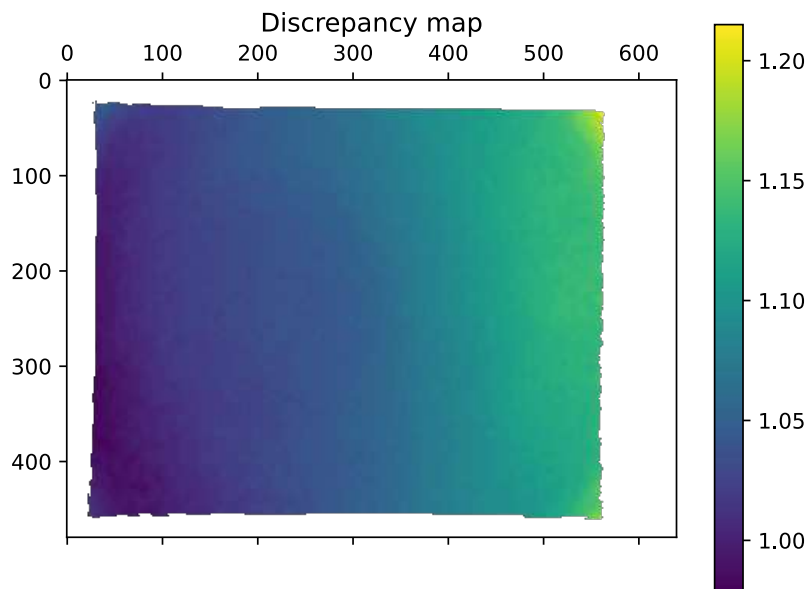


Figure 4.6: The resultant depth multiplier image from the plane-fitting approach.

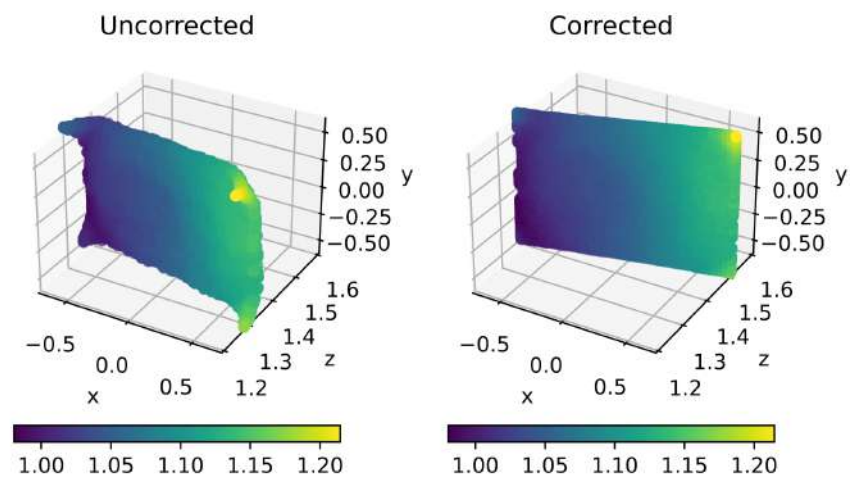


Figure 4.7: Left: Uncorrected depth measurement. Right: Same measurement with correction applied.

in the correction plot in Figure 4.9. This is due to the calibration process treating the laser scan data as ground truth without any filtering. This means that any noise in the laser scan is reflected directly onto the process results.

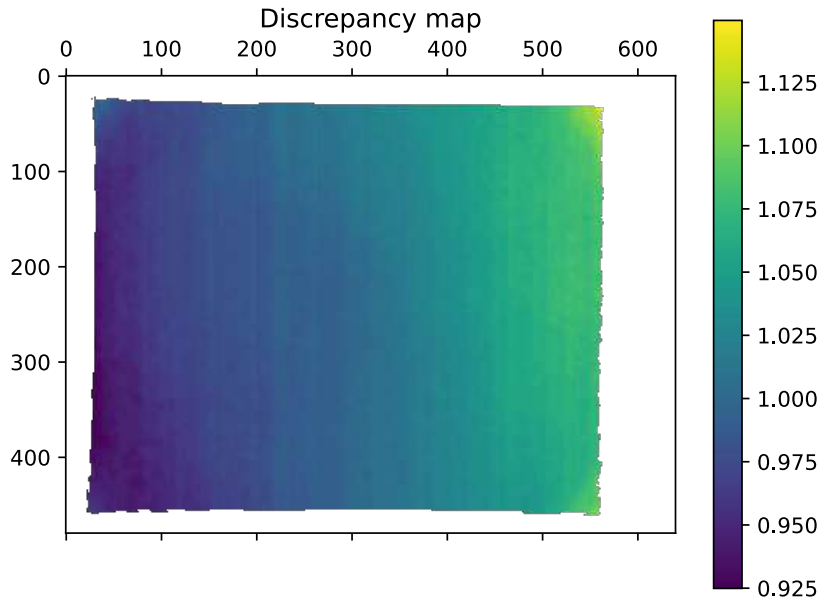


Figure 4.8: The resultant depth multiplier image from the extrusion approach.

4.2.3 Qualitative Comparison

We conduct an experiment to evaluate the resulting corrections from the two calibration methods presented. We position the robot in front of the whiteboard at different distances and record the resulting depth readouts when correction is applied using the calibration data obtained from the two methods. We quantify the recordings in terms of two metrics: Flatness (which we compute by taking the PCA of the pointcloud, and extracting the third highest explained variance ratio of the resulting components) and Accuracy (which we compute by comparing the depth measurement obtained for the top-center pixel of the image to depth measurements we make by hand). We collect data from 9 different distances ranging from 35cm to 210cm. The results are plotted in Figures 4.10 and 4.11.

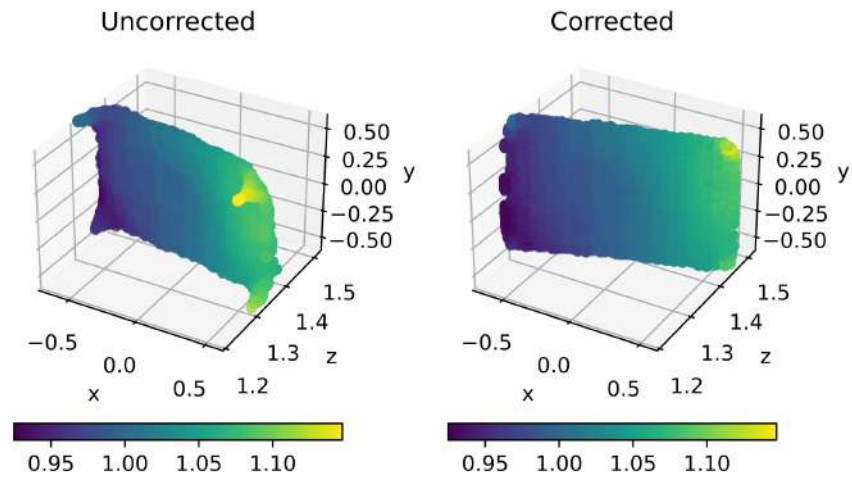


Figure 4.9: Left: Uncorrected depth measurement. Right: Same measurement with correction applied.

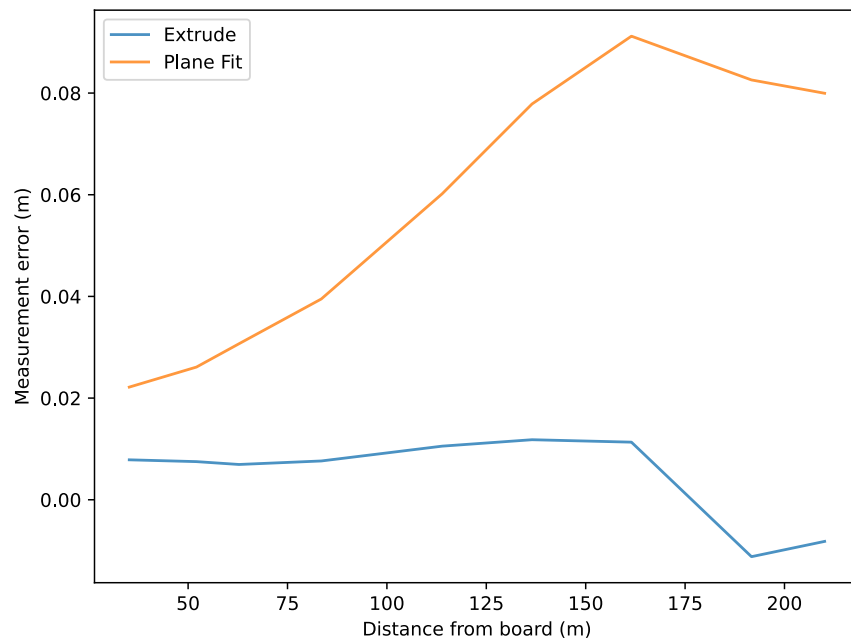


Figure 4.10: Depth measurement error for the top center pixel after correction with the extrusion method vs. the plane fitting method.

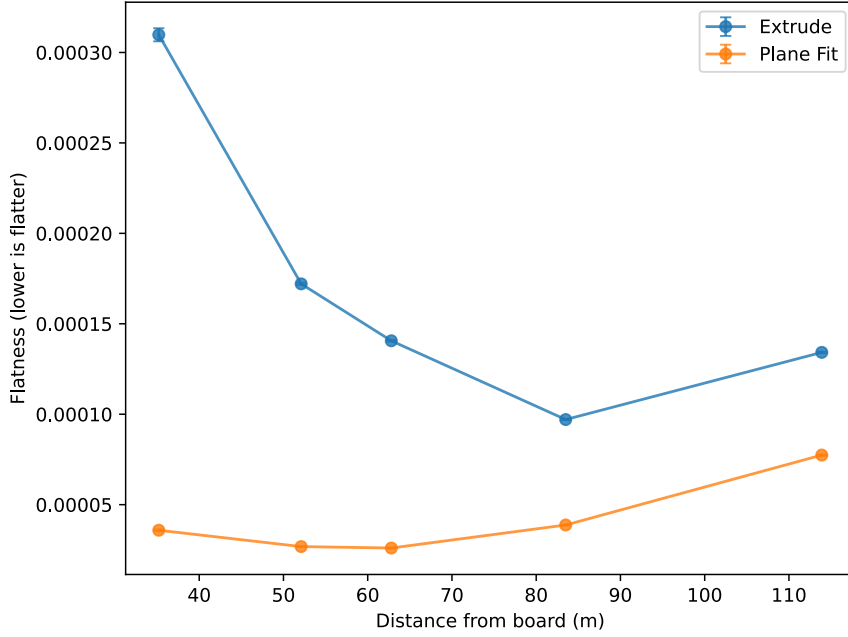


Figure 4.11: Measured flatness of whiteboard after correction with the extrusion method vs. the plane fitting method.

Note that Figure 4.11 only takes into account measurements up to 113cm. This is because from this distance onward the scene seen by the camera is not entirely flat due to the whiteboard's edges and other objects entering the frame. Looking at the measurement errors we observe that our extrusion-based calibration method yields more accurate measurements over all distances, while the plane-fitting method has large errors that increase with distance. We speculate that this may be caused by the linear regression process getting affected by laser measurement noise as distance increases. However, we note that the plane-fitting method can conserve the flatness of flat objects much better compared to the extrusion method. The poor performance of the extrusion method in this metric is expected and discussed in the previous chapter. As seen in Figure 4.9, the laser measurement noise directly affects the calibration process, and this can cause high amounts of jitter especially if the laser measurements are concentrated densely, as is the case with close-range measurements. This explains the lower amount of flatness for the extrusion process in close-range measurements.

4.2.4 Comparison and Conclusion

Comparing the two methods' recording processes, the main difference is in the existence of vertical stripes in the depth image resulting from the extrusion method. To create calibration coefficients, we use both methods to take 4 recordings of the wall at different distances ranging from 50cm to 4m.

In qualitative tests within the laboratory environment, we determined that at runtime the correction multipliers obtained from both methods seemed to yield a satisfactory pointcloud consistent with the real shape of the environment. We measured that the plane-fitting method can produce smoother measurements, but with accuracy errors, compared to the extrusion method which can produce more noisy, but more accurate measurements. We opt to use the coefficients calculated via the extrusion method in our project due to the high amounts of error in the plane-fitting method.

4.3 Qualitative Evaluation of the Depth Correction Module

In this section we provide corrected and uncorrected pointclouds of the whiteboard from various distances. For calibration we use the coefficients obtained via the extrusion method, as described in Section 4.2.

The accuracy of the pointcloud at close-range is very important. The system's drawing routine uses the depth mode of the board detection module when drawing, which fits a plane to the pointcloud and uses this plane to generate the drawing motion trajectory. Comparing the short-range performance of our rectifier as displayed in Figure 4.12, we note that the corrected readout is parallel to the LiDAR readout from the wall below. The offset between the pointclouds and the wall is because the board is offset from the wall it is attached to. We also notice that the vertical artifacts caused by the extrusion method are on display in this image. The small perturbances in the pointcloud may cause some problems when a plane is being fit to them and skew the board pose estimate angle slightly.

In Figure 4.13 showing a comparison of the mid-range performance of the correction module, we make similar observations to those for the short-range results. The distortions

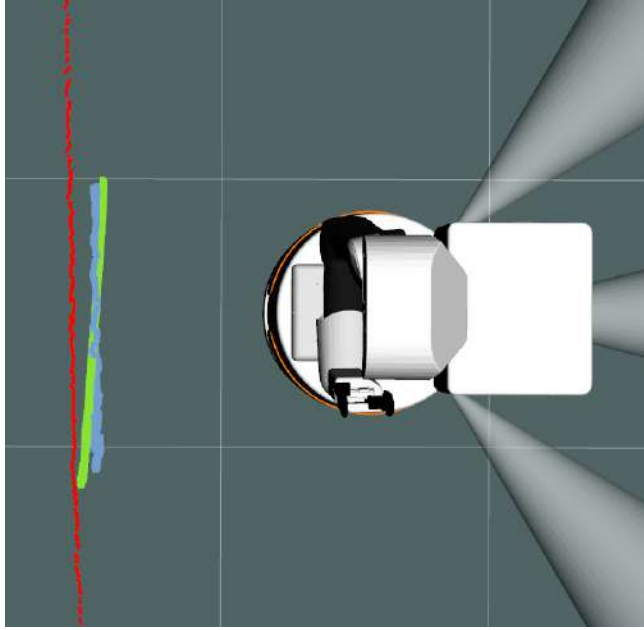


Figure 4.12: RViz screenshot showing uncorrected (green) vs corrected (blue) point-cloud of the board at 50cm distance.

tion present in the uncorrected pointcloud is much more prominent here. Our rectifier seems to correct for the extreme distortions in the corners of the image well. Since depth readouts in the medium and long range are used only for moving toward the board, they do not need to be too accurate. Figure 4.15 shows a side view comparing the corrected stream against the original.

We see the results for longer-range measurements in Figure 4.14. Two things notable here are the extreme distortions in the uncorrected pointcloud and the slight distortions in the corrected one. It's possible to make the observation that the corrected pointcloud is not perfectly parallel to the laser readout in this figure. This indicates that slight correction problems still exist. In practice, this is not too important since our long-range uses of the pointcloud streams does not demand high accuracy.

To conclude, we note that the distortion of the pointcloud appears to increase as TIAGo gets farther away from the board. Further work may investigate how the distortion pattern relates to distance. If the growth is linearly related, a single depth multiplier image would be enough for calibration without the need for linear interpolation.

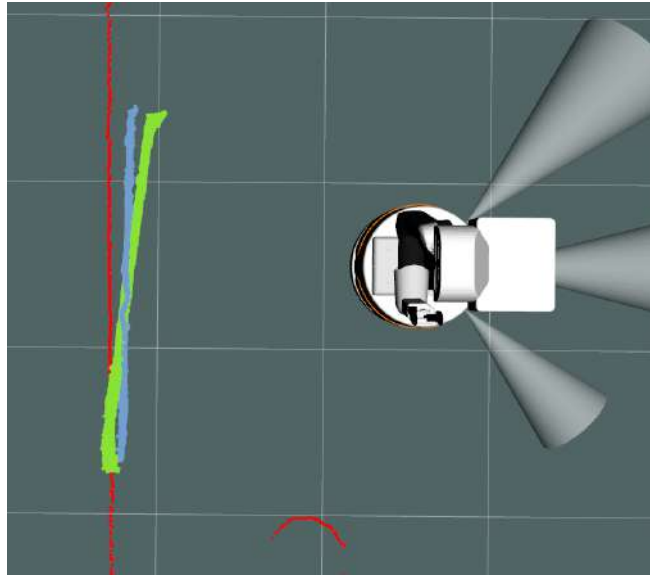


Figure 4.13: RViz screenshot showing uncorrected (green) vs corrected (blue) point-cloud of the board at 104cm distance.

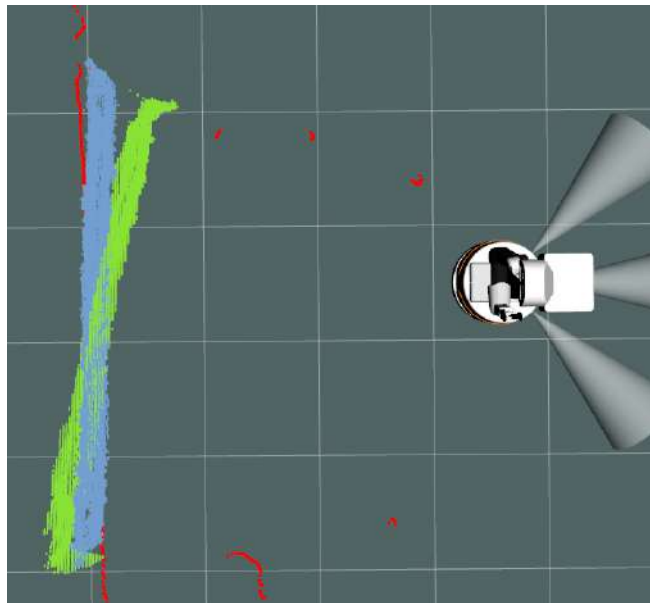


Figure 4.14: RViz screenshot showing uncorrected (green) vs corrected (blue) point-cloud of the board at 232cm distance.

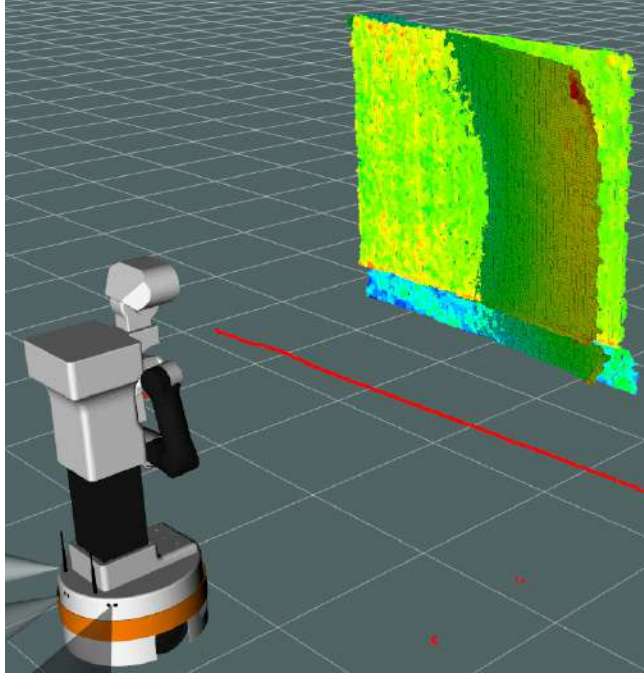


Figure 4.15: RViz screenshot showing side view of uncorrected (dark) vs corrected (light) pointcloud of the board at 170cm distance.

While our correction module does not yield perfect results especially at longer ranges as shown by our qualitative evaluation, the performance of the system seems to be high enough that the module can be used without issue for the purposes of implementing our drawing system.

4.4 Qualitative Comparison of Board Detection Methods

In this section of the work, we aim to illustrate and qualitatively compare the performance of the various board detection and pose estimation approaches we proposed.

Figure 4.16 depicts the expected case in board collision box generation. Once the board plane has been estimated, the board dimensions stored in the program are used to construct an approximate collider for the board. While the board is offset slightly from the wall it is attached to, we extend the collision box downward from the board plane itself in an effort to increase our safety margins.

Figure 4.17 illustrates an advantage of the RANSAC approach compared to the least-

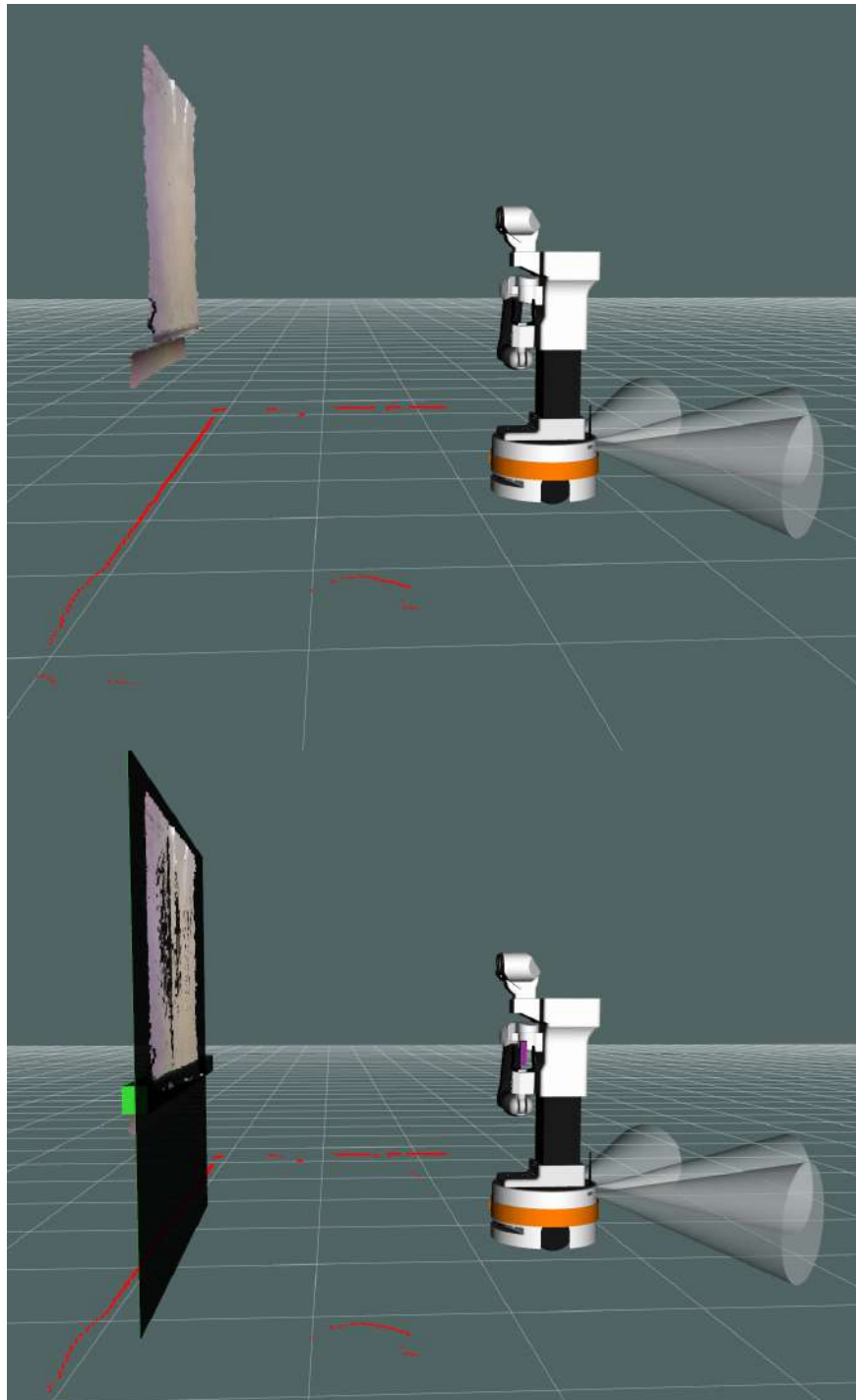


Figure 4.16: RViz screenshot illustrating board detection using the depth camera and RANSAC. Top: Pointcloud only, Bottom: Pointcloud and bounding box. Note the extra collision volume for the board lip.

squares method. Since RANSAC ignores spurious measurements, in cases where the board is not the only thing in view the plane estimation does not suffer. This is also useful when the robot’s arm enters the field-of-view when drawing. RANSAC counts it as a spurious measurement while for total error minimization methods like least-squares this can negatively affect the quality of the results.

Figure 4.18 shows a successful detection of the whiteboard using the YOLO model. The mask generated by the model can be used by the depth-based pose estimation model, which can use the mask to filter the pointcloud and run its plane-fitting algorithms on the resulting data. While YOLO was able to provide high quality masks for the whiteboard in most cases, early iterations would also flag false positives often, an example of which can be seen in Figure 4.19. This was fixed by expanding the variety of images in the dataset.

In Figure 4.20 and Figure 4.21 sample board pose estimation results using the ArUco module for one marker and two markers respectively are shown. The pose estimate for the single-marker case is slightly skewed. The reason for the better pose estimate in the two-marker case is because the points on the plane are farther apart from each other, which makes estimating the pose easier compared to the single-marker case where the relatively low image resolution can adversely effect the pose estimates. In practice, the poor estimates in single-marker cases can cause significant problems if their pose estimates are used to determine where to go to stand in front of the board. The movement target of the robot ends up being considerably skewed, causing problems for downstream tasks. Because of this, we do not use single-marker pose estimates for servoing. Instead, we use the information from one marker to attempt to see both markers at once by repositioning our camera.

4.5 Quantifying Pose Estimation Accuracy

In order to evaluate the accuracy of the pose estimates generated by the various board pose estimators we created, we formulate an experiment. A challenge we face in measuring the accuracy of the estimates is that we lack a ground-truth source of board pose data. To act as the baseline, we try attaching a large ArUco marker with an

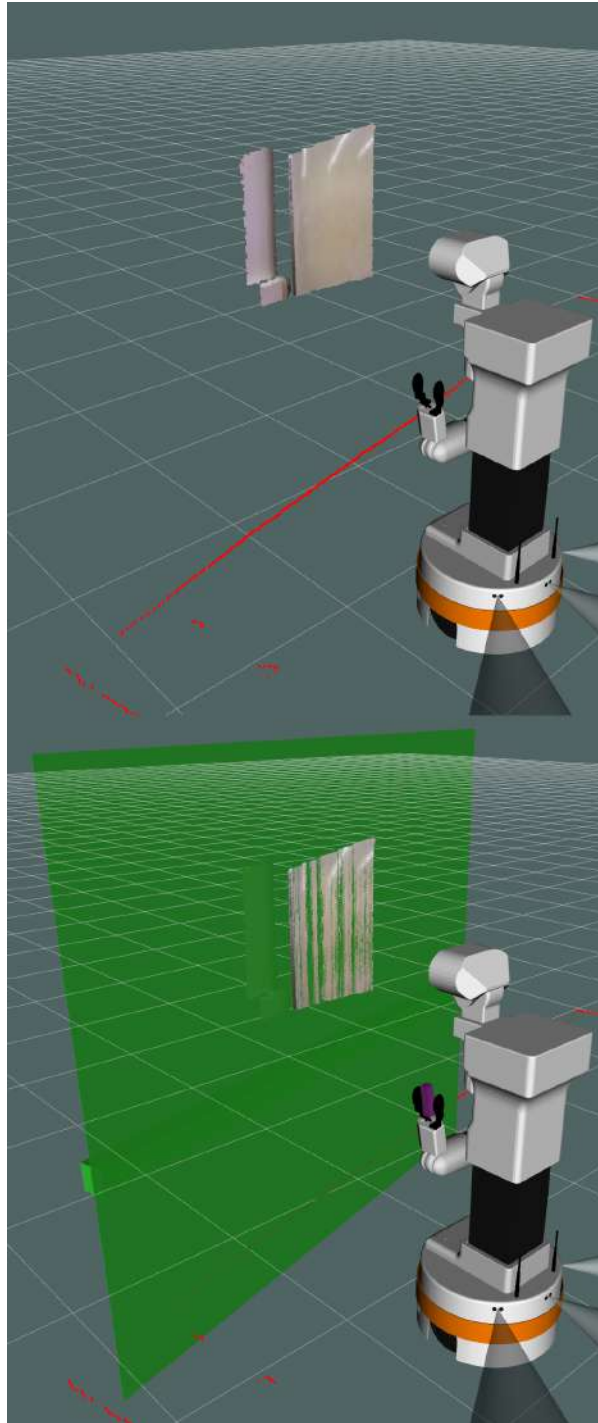


Figure 4.17: RViz screenshot illustrating board detection using the depth camera and RANSAC. Top: Pointcloud only, Bottom: Pointcloud and bounding box.

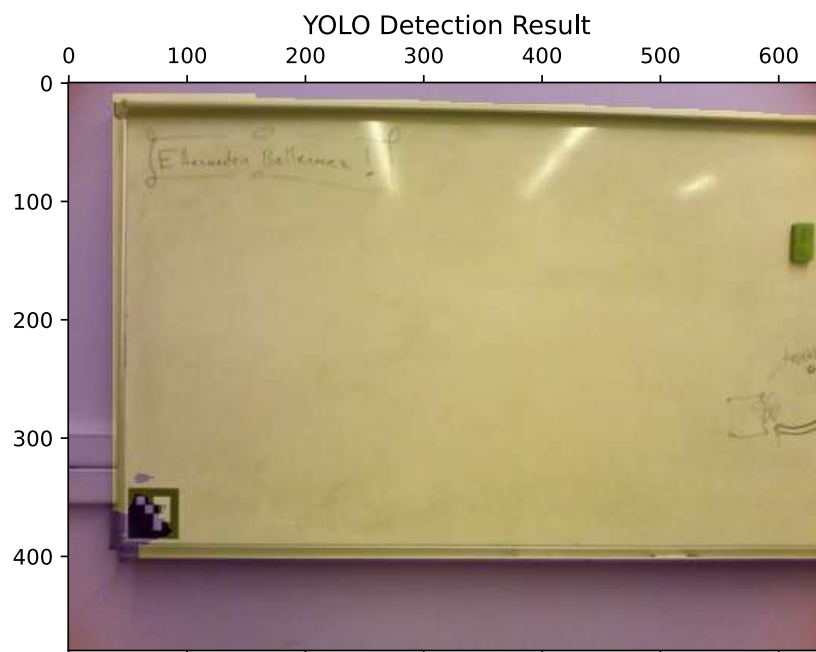


Figure 4.18: A successful board detection using the YOLO model. Yellow denotes the proposed mask.

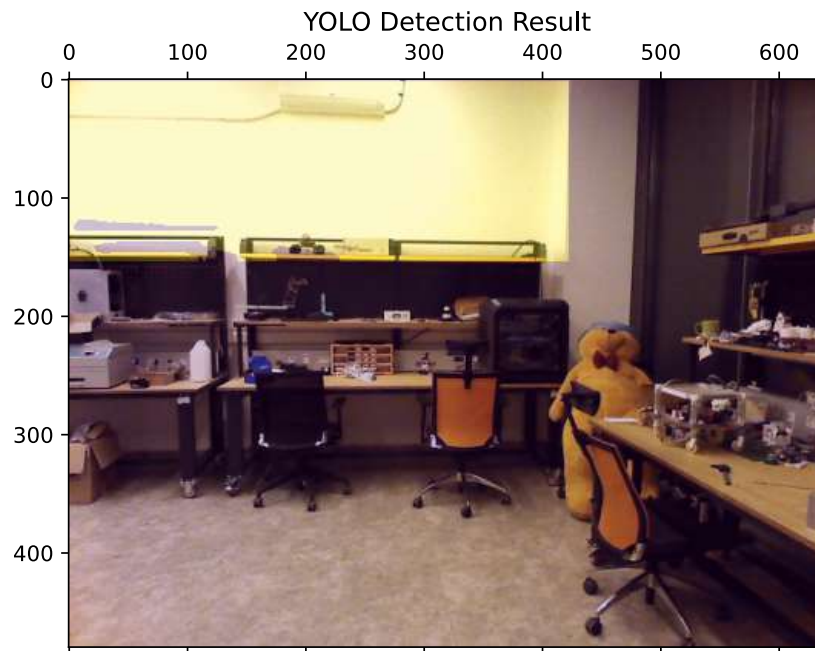


Figure 4.19: A false positive board detection from the YOLO model. Yellow denotes the proposed mask.

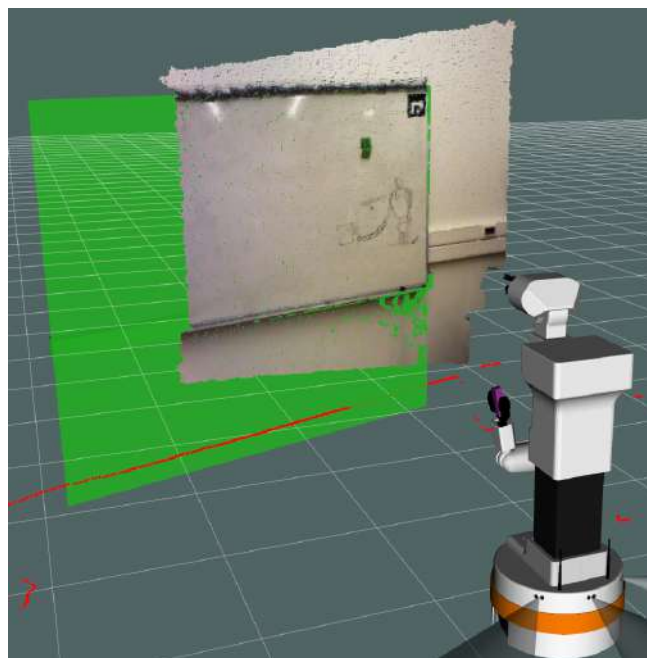


Figure 4.20: RViz screenshot illustrating poor board detection result when only one ArUco marker is in view.

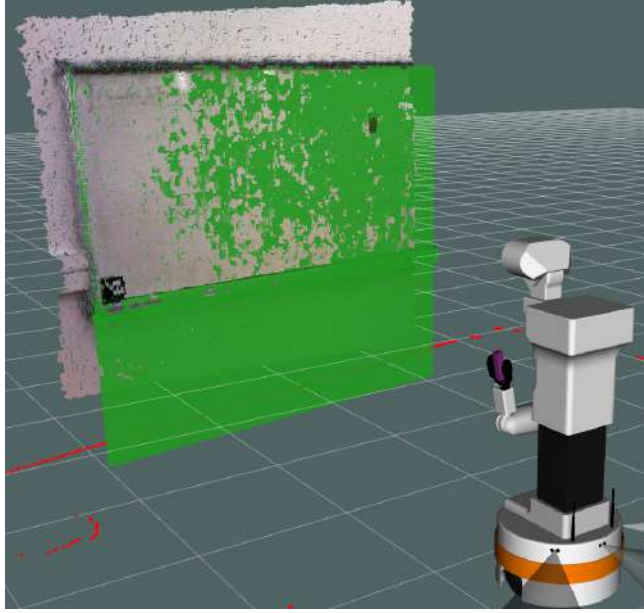


Figure 4.21: RViz screenshot illustrating good board detection result when both ArUco markers are in view.

edge length of 19cm to the middle of the board, and introduce a new pose estimator called the large ArUco submodule which uses this large marker to estimate the board's orientation. Since this marker is larger than the ones we use for the ArUco submodule, we expect it to provide us with more reliable measurements. We then compare the pose estimates we obtain from our YOLO and ArUco marker submodules with the estimate resulting from the larger ArUco marker. The whiteboard configuration is presented in Figure 4.22. However, we notice that the estimates obtained from this single-marker source are considerably worse than the two submodules we compare it against upon visual inspection. Therefore we do not use the data obtained from that source for analysis here. However, we include the data in the appendix in Section A.2.

We position the robot at various distances from the board and record the angular distance between the position estimates. We calculate this distance with the following formula for two rotation matrices R_1 and R_2 :

$$\| \log R_1 R_2^T \| \in [0, \pi] \quad (4.1)$$

For each distance, the measurements are repeated 50 times. The results we obtained



Figure 4.22: Whiteboard with the large ArUco marker attached to the middle.

are listed in Table 4.1. A discussion of these results is provided toward the end of this section.

Distance to board (m)	YOLO vs. Depth (rad)	
	mean	std. dev.
5	0.205	0.058
4	0.072	0.038
3	0.055	0.014
2	0.076	0.008

Table 4.1: Pose estimate angular differences between the YOLO submodule and the ArUco submodule.

In addition, we collect data for the case where the robot can only see one of the ArUco markers in order to analyze how the number of detected markers affects the estimate accuracy. At each testing distance, we record measurements with the robot positioned to have either both markers in view or one marker in view, as pictured in Figures 4.24 4.23. The results of these tests are shown in Table 4.2.

The plot of the measurements we obtained can be found in Figure 4.25. A challenge we face when interpreting this data is the lack of a ground-truth measurement since our attempt at providing one did not result in a reliable benchmark. It can be seen that



Figure 4.23: The TIAGo robot positioned so that both markers are in view.



Figure 4.24: The TIAGo robot positioned so that one marker is in view.

Distance to board (m)	Double ArUco vs. YOLO (rad)		Single ArUco vs. YOLO (rad)	
	mean	std. dev.	mean	std. dev.
5	0.205	0.058	0.225	0.123
4	0.072	0.038	0.333	0.205
3	0.055	0.014	0.091	0.024
2	0.076	0.008	0.063	0.006

Table 4.2: Pose estimate angular differences between the ArUco submodule and the YOLO submodule. "Single" describes the case where the ArUco submodule can see only one marker, while "double" describes the case where the submodule can see both markers.

in close distances the ArUco and YOLO methods provide similar estimates. However, at longer ranges, we observe significant divergences. For the case where the robot can see both markers, we note that the agreement between methods extends to the 4m mark. At 5 meters, we observe noise in the depth data, which corrupts the YOLO estimates, and occasional offsets and jumps in the ArUco estimates, leading to a high amount of orientation error between the estimates. For the single marker case, we observe a high case of divergence in the estimates at distances above 4m. Since the length of the corners of the marker observed get smaller and smaller in the camera frame, the pose estimates made from the discretized pixel-length measurements of these corners get more inaccurate. Surprisingly, we see a slight decrease in the angle difference for the single marker case at 5m, which we attribute to the YOLO submodule starting to show inaccuracies as well due to depth measurement errors. The findings obtained regarding the inaccuracy of the single-marker case in this experiment resulted in the addition of mechanisms to ensure that estimates from single-marker detections would not be used for navigation of the robot in the operating algorithm as described in Section 3.7.

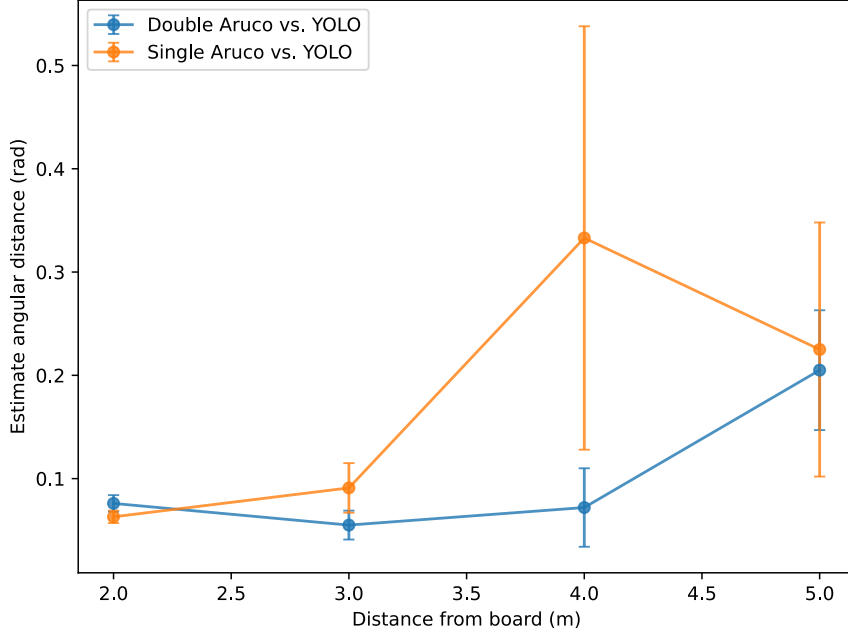


Figure 4.25: Board pose estimate angular differences of the ArUco submodules in the single marker and double marker cases in comparison to the YOLO submodule.

4.6 Evaluation of Probabilistic Motion Planners

In this section, we aim to quantify the reliability of the probabilistic motion planner we evaluated for the motion control module of our system, responsible for controlling the arm to move it to the board collision-free and draw things on the board. In attempting to use the MoveIt ROS library supplied with the robot for the drawing task, we noticed that the RRT-Connect algorithm in its default configuration would be highly sensitive to robot pose with respect to the board, and would occasionally fail to provide solutions to our motion planning problems. Such failures are very disruptive to operation since there wasn't an easy-to-implement automated method of mitigating such errors, and we would have to iteratively tweak the robot's pose to find a configuration where the planner could successfully output a motion plan. Since the motion planning occurs after a long string of operations locating and moving toward the board, a failure at this point is time-consuming to recover from. In order to illustrate the sensitivity to pose of the probabilistic planner, we formulate an experiment where we position the robot at different distances from the board and run our motion planner to unfurl the arm and draw a square. We run the planner with a 2 second

time-out 100 times and record its success rate.

The results for the unfurling are shown in Table 4.3. We note that in the region between 52cm and 98cm the robot can successfully find plans for the robot. As the robot gets close to the board, we observe that the plans involve unnatural movements involving the arm doing near-collision passes with itself and ending in an elbow-up configuration. Considering the eventual end-goal of having the robot interact with humans, we see this behavior as undesirable. Therefore when locating the robot in regular operation, we command the robot to stay at a distance of approximately 80cm from the board, which results in natural movements and a less imposing, elbow-down resting pose. The region in which the probabilistic planner is reliable is not too small, hence for unfurling movements the probabilistic planner is usable. Nevertheless we implement an alternative, pre-recorded approach as described in Section 3.5.2 to ensure unnatural-looking plans are not generated and executed.

Distance (cm)	Success Rate
40	0
48	0
52	100
60	100
80	100
98	0

Table 4.3: Success rate of RRT-Connect at planning the arm-unfurling motion at different distances.

For the drawing movement we instruct the robot to unfurl its arm and draw a square with the cartesian planning interface of RRT-Connect. Since we do not require cartesian interpolation of the end-effector for the unfurling motion, we only use this interface for the drawing motion itself. We use the same procedure to obtain the success rate for 100 tries. We note that we cannot get any successful plans to draw a 10cm wide square, which is the shape we use for most evaluations in this thesis. We instead use a 5cm wide square, which results in Table 4.4. Here the region of reliable evaluations is much more limited, localized within a region from 58cm to 66cm. We

note that a 23% chance of failure is highly detrimental to the successful operation of the system. Additionally, we observe that the regions in which the planner can find a solution are limited to those where the unfurling may result in unnatural motion since the robot is so close to the board. In contrast, the RRMC submodule we implement can draw the 10cm wide square reliably in the 80cm distance we determine to produce good results for unfurling. Not using the cartesian interface results in the arm interpolating freely between the waypoints of the drawing trajectory, which doesn't result in straight lines. Increasing the amount of waypoints is equivalent to using the cartesian interface, where the failure rate becomes high. Due to the limited distance range and limited drawing range in which the probabilistic planner can output motion plans reliably, we opt to use the RRMC submodule for our system. We speculate that it's possible to increase the operating region in which the probabilistic planner we use can reliably generate motion plans by tuning the search parameters of the planner.

Distance (cm)	Success Rate
48	0
52	72
58	77
61	100
66.5	0
71	0

Table 4.4: Success rate of RRT-Connect at planning the arm-unfurling and drawing motion at different distances.

4.7 Comparison of Resolved-Rate Motion Controllers

In this experiment we do a parameter sweep over various resolved-rate motion controller settings in order to understand how they affect the drawing performance of the robot. The qualities we desire in our drawing routine are accuracy and speed. In the next section we define appropriate metrics to quantify these qualities.

4.7.1 Formulation of Metrics

In order to measure the drawing speed of the robot, we measure the time it took for the robot to finish drawing a given command, starting from the drawing-ready arm configuration.

To measure the accuracy, we define a procedure to calculate the total weighted mean squared of path error. First we record the robot's end-effector at each iteration of the resolved-rate motion controller. To compare how far away each recorded point is from the point it should be in, we use linear interpolation: While the robot is executing a draw command from point p_0 to p_1 , we record the following: $\hat{p}_{0,0}, \hat{p}_{0,1}, \hat{p}_{0,2}, \hat{p}_{1,0}$. Each point $\hat{p}$ has its recording time stored as well: $t_{0,0}, t_{0,1}, t_{0,2}, t_{1,0}$. We know that $\hat{p}_{0,0}$ corresponds to the point p_0 , and we can compute their difference. For the interim points, we linearly interpolate the commanded points to obtain the point where the end-effector should be at a given point in time. For example, the corresponding commanded point to $\hat{p}_{0,1}$ is calculated as follows:

$$p_{0,1} = \frac{t_{0,1} - t_{0,0}}{t_{1,0} - t_{0,0}}(p_1 - p_0) + p_0 \quad (4.2)$$

After computing the corresponding commanded points for each recorded point, we calculate the mean squared error in all 3 dimensions separately. We then take the weighted average of these 3 error values, where the value corresponding to error in z is downweighted because our drawing operation is not sensitive to changes in z due to the mechanical compliance of the gripper. This gives us our score.

4.7.2 Experiments and Results

To conduct the experiments, we write a function that takes in a list of controllers and corresponding parameters, and uses each of them to draw a square on the whiteboard while computing our metrics: Score and Time. The settings that were tested along with their performance metrics are listed in Table 4.5. Parameter sweeps are grouped together and groups are separated by horizontal lines. Plots showing the recorded trajectories against their commanded counterparts can be seen in Figure 4.26 and Figure 4.27. A discussion of the results obtained is given in the next subsection.

Name	Samples per Waypoint	Primary Threshold	Secondary Threshold	K_P	K_I	K_D	Score	Time (s)
PID	2	0.03		1.5	0.3	-0.01	0.252	16.28
PID	5	0.03		1.5	0.3	-0.01	0.247	19.31
PID	1	0.03		2	0.3	-0.01	0.251	10.26
PID	1	0.03		1	0.3	-0.01	0.245	13.00
PID	1	0.03		1.5	0	-0.01	0.240	11.60
PID	1	0.03		1.5	0.5	-0.01	0.235	11.35
PID	1	0.03		1.5	0.3	-0.1	0.236	11.48
PID	1	0.03		1.5	0.3	-0.001	0.236	10.85
PID	1	0.01		1.5	0.3	-0.01	0.207	34.46
PID	1	0.05		1.5	0.3	-0.01	0.229	8.85
SkippingPID	1	0.03	0.05	1.5	0.3	-0.001	0.230	11.09
SkippingPID	2	0.03	0.05	1.5	0.3	-0.001	0.227	11.50
SkippingPID	5	0.03	0.05	1.5	0.3	-0.001	0.230	13.15
SkippingPID	1	0.02	0.05	1.5	0.3	-0.001	0.232	17.96
SkippingPID	1	0.01	0.03	1.5	0.3	-0.001	0.235	35.23

Table 4.5: Calculated accuracy scores and runtimes for various resolved-rate motion controller configurations.

```
<class 'planner.ProbPID'>{'samples_per_interp': 5,  
  'threshold': 0.03, 'Kp': 1.5, 'Ki': 0.3, 'Kd': -0.01}  
SCORE:0.247  
TIME:19.31
```

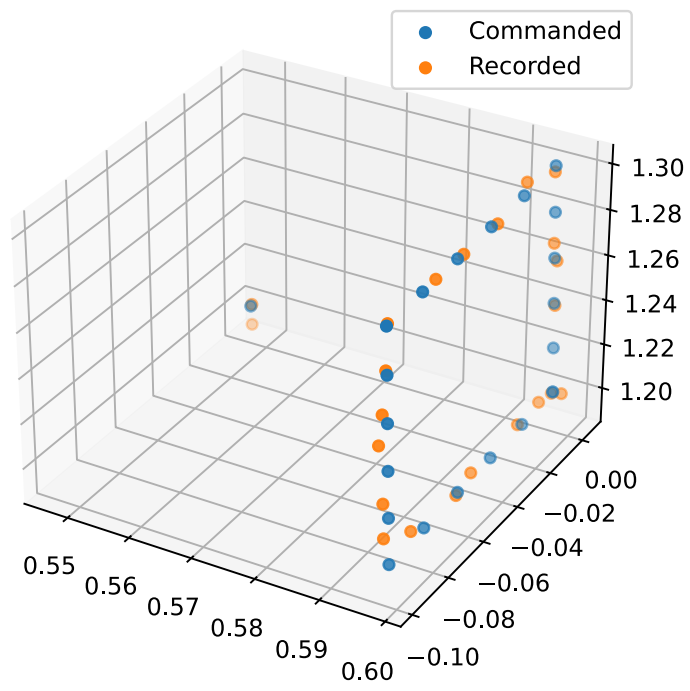


Figure 4.26: PID controller trajectory plotted against commanded waypoints.

```
<class 'planner.ProbSkippingPID'>{'samples_per_interp': 5,  
  'primary_threshold': 0.03, 'secondary_threshold': 0.05,  
  'Kp': 1.5, 'Ki': 0.3, 'Kd': -0.001}  
SCORE:0.230  
TIME:13.15
```

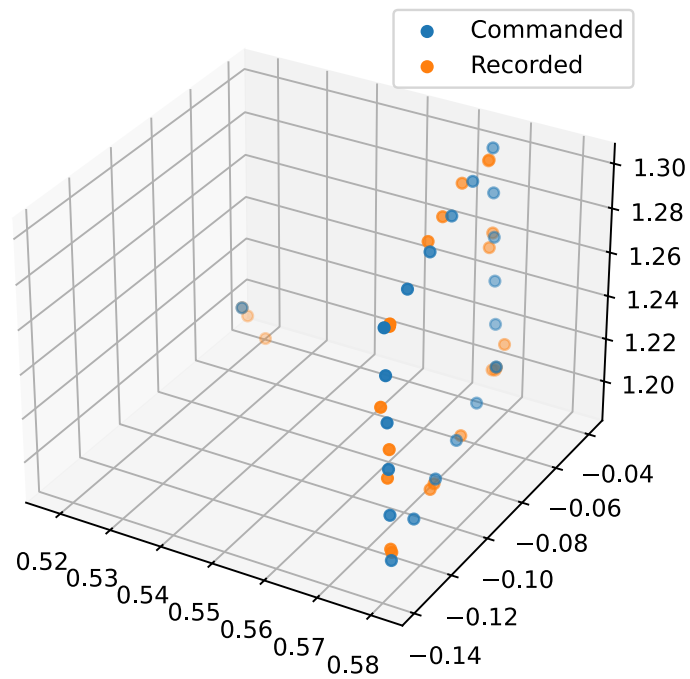


Figure 4.27: SkippingPID controller trajectory plotted against commanded waypoints.

4.7.3 Discussion

In the first group of experiments we can observe the effect of changing the number of samples per waypoint. As expected, the score decreases (indicating less error) with an increased number of sub-waypoints, indicating that the trajectory is followed more accurately, while the time taken increases as a tradeoff. In the fifth group of experiments, we change the number of samples while using the skipping PID controller (the skipping PID controller is explained in Subsection 3.5.3). The change in accuracy is not as clear here, but a slowdown with increasing samples is observed.

The second group of experiments show that decreasing K_P has a similar effect, increasing the accuracy but also increasing the time taken.

The third and fourth groups of experiments study changes in K_I and K_D respectively, but the results are too close together to make a conclusive statement.

The fifth and seventh groups of experiments investigate the effect of changing the threshold. Making the threshold more strict seems to impose considerable cost on the drawing time, with slight benefits to accuracy.

Comparing the results for the eighth and eleventh rows, we can comment on the effect of the skipping PID controller, which seems to be negligible.

To summarize, increasing the number of subsamples or decreasing the controller gain can increase accuracy at the cost of runtime, and setting the threshold to a value that's too strict can cause considerable slowdowns. The rest of the parameters' effects seem to be negligible. The results indicate that a controller with a low amount of samples per waypoint (preferably 1, i.e. no sub-waypoints) and a sufficiently lenient threshold performs better than the alternatives.

4.8 Measurement of Arm Backlash

We note the existence of backlash in the robot's end-effector during motions. With this experiment we aim to quantify the amount of backlash in the robot arm. To achieve this, we position the robot in front of the board and move the arm to its draw-

ing configuration such that the marker is making contact with the board, as pictured in Figure 4.28. With the arm motors in position control mode, we lightly nudge the end-effector in various directions and measure the resulting deflections from the marks left.

Even without the effects of backlash, the arm has low enough impedance that one can apply a force on the end effector to deflect the motors from their target positions significantly. In order to measure the backlash only, we apply very light forces on the end-effector and stabilize the axes of the arm that were not found to cause backlash on prior evaluations.

We determine that axes 5 and 6 of the arm have backlash. These axes are responsible for rotation of the wrist, and their axes of rotation are perpendicular to each other. Consequently, their effects on the end-effector are also perpendicular and can be analyzed separately.

Figure 4.29 and Figure 4.30 show the gear transmissions causing the backlash in axes 5 and 6 respectively. We engage axis 5 to obtain the mark shown in Figure 4.31, and axis 6 to obtain the mark shown in Figure 4.32. The displacements we obtain are tabulated in Table 4.6. These measurements indicate some inherent limitations in the arm's precision.



Figure 4.28: The TIAGo with its marker touching the whiteboard. While the robot is in this pose, we apply forces to the end-effector by hand to measure deflections.

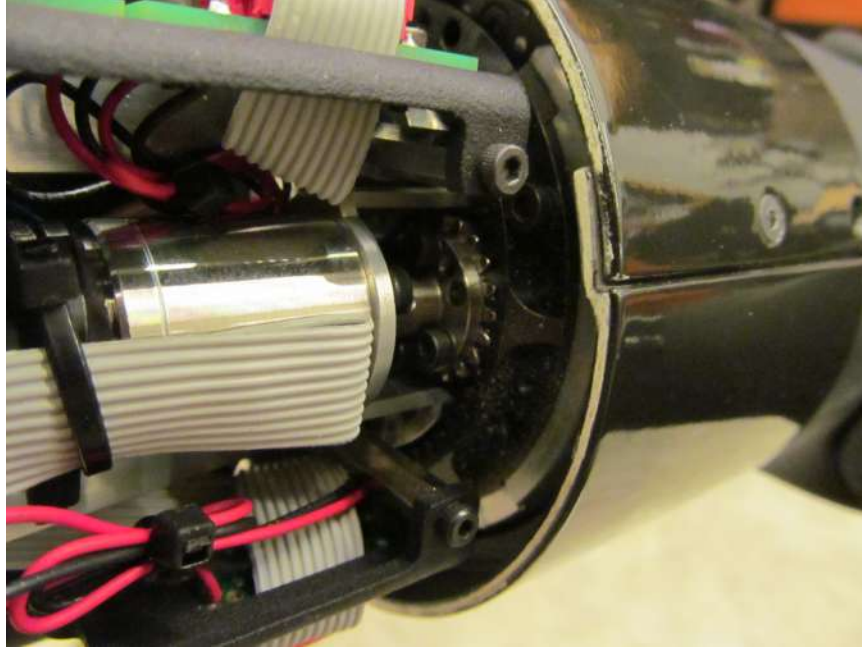


Figure 4.29: The gear transmission in the fifth axis of the robot arm. The internal gear visible in the picture meshes with the external gear painted black.

Axis	Horizontal Length (cm)	Vertical Length (cm)	Lengthwise Deflection (cm)
5	1.4	1.0	1.72
6	0.9	1.0	1.34

Table 4.6: Length measurements of the backlash deflection marks. Lengthwise deflection calculated via Pythagoras' theorem.

Additionally, we formulate an experiment to visualize the backlash present in the robot. We use the robot to draw the same square clockwise and counterclockwise to observe the differences in the resulting trajectories. This gives us an impression of the hysteresis-like nonlinearities present in the arm. For each direction, we draw the same square twice to show that the deformations are consistent. We use a PID RRMC scheme with no interpolated sub-waypoints, a threshold of 0.03, $K_p = 1.5$, $K_i = 0.3$ and $K_d = -0.01$. The results can be seen in Figure 4.33. Note that the small "tail" seen in the bottom left of the square is due to end-effector interpolation when drawing the same shape again. We observe that the deviations occurring within the repeats in either direction are unchanging.



Figure 4.30: The gear transmission in the sixth axis of the robot arm.

4.9 Operating Algorithm Reliability

We evaluate the reliability of the full system by running the operating algorithm described in Section 3.7 multiple times and record the success rate of the execution. In this experiment we run the algorithm 15 times in a row and count a run as being successful if the robot successfully detects the board, moves to it and draws a complete square before terminating. Out of 15 runs, our system succeeds in this task a total of 9 times. In one of the successful runs the robot arm briefly collides with the robot's body due to a control error during the unfurling of the arm. We attribute the 6 failures to errors in the built-in navigation stack. In 5 of the runs the robot's localizer failed to track the robot's location while the robot was moving toward the board, leading to the robot veering off course. In the other run resulting in failure, the robot wrongly determined that it was within an obstacle during the board detection stage, rendering the robot unable to move even though it estimated the board pose successfully.



Figure 4.31: The mark created by the deflections from the fifth axis.



Figure 4.32: The mark created by the deflections from the sixth axis.



Figure 4.33: Square drawn twice clockwise on the left image, and drawn twice counterclockwise on the right.

CHAPTER 5

CONCLUSIONS

In this thesis we built a software system for the TIAGo robot that can detect and move to whiteboards, and then draw arbitrary paths on them. We started the project motivated by the way humans use whiteboards as a mode of communication. In creating a holistic system for drawing on whiteboards, we were able to provide an outlook on the challenges faced in designing for an everyday robotics problem using popular classical approaches, giving some insight as to where the current solutions succeed and in which cases they fall short. We decomposed our problem into different subproblems and implemented and evaluated various solutions to address them. For the problem of board detection we investigated multiple sensor modalities, exploring depth-based approaches, data-driven object recognition models and fiducial markers. Spurred by errors in our depth camera data, we investigated depth camera calibration methods and implemented our custom variations on pre-existing calibration algorithms. We prototyped different approaches to ensuring stable pen contact when drawing, designing and manufacturing a mechanical solution to our problems. For the problem of motion planning of the drawing procedure we evaluated various probabilistic motion planners and faced issues with consistency of output. We explored alternative methods to find solutions that were a better fit to our problem. While developing and implementing the software, we used a highly dynamic and modular development methodology that allowed us to develop with fast iteration times. We tested out our module implementations' performances in real-life cases. We were able to discover various challenges that one may face in the process of writing robot software, and also highlight the limitations of some popular approaches in the context of real-life applications.

5.1 Future Work

Future avenues of work building on this thesis may include adding a layer of interactive behaviors to the robot to enable collaborative drawing. This layer may be comprised of any number of different data-driven models. The layer could be as simple as writing words in response to the whiteboard contents generated from a language model, or it could involve more complex configurations whereby a diffusion model is used with inpainting to complete drawings, the output of which is vectorized and executed as a path. Improvements on system reliability could always be pursued, be it by tweaking the system architecture or by implementing new methods to solve the existing set of subproblems.

REFERENCES

- [1] M. Bajracharya, J. Borders, R. Cheng, D. Helmick, L. Kaul, D. Kruse, J. Leichty, J. Ma, C. Matl, F. Michel, C. Papazov, J. Petersen, K. Shankar, and M. Tjersland, “Demonstrating Mobile Manipulation in the Wild: A Metrics-Driven Approach,” in *Robotics: Science and Systems XIX*, Robotics: Science and Systems Foundation, July 2023.
- [2] Z.-E. Chebab, J.-C. Fauroux, N. Bouton, Y. Mezouar, and L. Sabourin, “Autonomous Collaborative Mobile Manipulators: State of the Art,” in *TrC-IFTToMM*, (Izmir, Turkey), 2015.
- [3] D. Song, E. Lim, J. Park, M. Jung, and Y. J. Kim, “TSP-Bot: Robotic TSP Pen Art Using High-DoF Manipulators,” in *2024 21st International Conference on Ubiquitous Robots (UR)*, (New York, NY, USA), pp. 100–105, IEEE, June 2024.
- [4] D. Song, T. Lee, and Y. J. Kim, “Artistic Pen Drawing on an Arbitrary Surface Using an Impedance-Controlled Robot,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, (Brisbane, QLD), pp. 4085–4090, IEEE, May 2018.
- [5] A. Haddadi, *Robotic Drawing on a 3D Object*. Diploma thesis, TU Wien, Vienna, Nov. 2018.
- [6] T. Weingartshofer, C. Hartl-Nesic, and A. Kugi, “Automatic and Flexible Robotic Drawing on Complex Surfaces With an Industrial Robot,” *IEEE Transactions on Control Systems Technology*, vol. 32, pp. 1602–1615, Sept. 2024.
- [7] L. Chen, A. Naceri, A. Swikir, S. Hirche, and S. Haddadin, “Autonomous and Teleoperation Control of a Drawing Robot Avatar,” July 2024.
- [8] D. Berio, S. Calinon, and F. F. Leymarie, “Learning dynamic graffiti strokes with a compliant robot,” *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3981–3986, Oct. 2016.

- [9] J. Song, M. Kang, and N. Kwak, “SketcherX: AI-Driven Interactive Robotic drawing with Diffusion model and Vectorization Techniques,” Sept. 2024.
- [10] T. Wang, W. Q. Toh, H. Zhang, X. Sui, S. Li, Y. Liu, and W. Jing, “RoboCo-Draw: Robotic Avatar Drawing with GAN-Based Style Transfer and Time-Efficient Path Optimization,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 10402–10409, Apr. 2020.
- [11] M. Abrini, M. Auvray, and M. Chetouani, “Humans’ Spatial Perspective-Taking When Interacting with a Robotic Arm,” in *2023 32nd IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, (Busan, Korea, Republic of), pp. 533–540, IEEE, Aug. 2023.
- [12] B. Waarsing, M. Nuttin, and H. Van Brussel, “Behavior-based mobile manipulation inspired by the human example,” in *2003 IEEE International Conference on Robotics and Automation (Cat. No.03CH37422)*, vol. 1, pp. 268–273 vol.1, Sept. 2003.
- [13] Y. Lin, J. Guo, Y. Chen, C. Yao, and F. Ying, “It Is Your Turn: Collaborative Ideation With a Co-Creative Robot through Sketch,” in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI ’20, (New York, NY, USA), pp. 1–14, Association for Computing Machinery, Apr. 2020.
- [14] C. Gomez Cubero, M. Pekarik, V. Rizzo, and E. Jochum, “The Robot is Present: Creative Approaches for Artistic Expression With Robots,” *Frontiers in Robotics and AI*, vol. 8, July 2021.
- [15] A. Kotani and S. Tellex, “Teaching Robots To Draw,” in *2019 International Conference on Robotics and Automation (ICRA)*, (Montreal, QC, Canada), pp. 4797–4803, IEEE, May 2019.
- [16] T. Gervet, S. Chintala, D. Batra, J. Malik, and D. S. Chaplot, “Navigating to objects in the real world,” *Science Robotics*, vol. 8, p. eadf6991, June 2023.
- [17] R. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal on Robotics and Automation*, vol. 2, pp. 14–23, Mar. 1986.

- [18] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, pp. 381–395, June 1981.
- [19] W. Zimmer, E. Ercelik, X. Zhou, X. J. D. Ortiz, and A. Knoll, “A Survey of Robust 3D Object Detection Methods in Point Clouds,” Mar. 2022.
- [20] G. Jocher and J. Qiu, “Ultralytics YOLO11,” 2024.
- [21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (Las Vegas, NV, USA), pp. 779–788, IEEE, June 2016.
- [22] F. Hermens, “Automatic object detection for behavioural research using YOLOv8,” *Behavior Research Methods*, vol. 56, pp. 7307–7330, May 2024.
- [23] C. Zhang and Z. Zhang, “Calibration Between Depth and Color Sensors for Commodity Depth Cameras,” in *Computer Vision and Machine Learning with RGB-D Sensors* (L. Shao, J. Han, P. Kohli, and Z. Zhang, eds.), pp. 47–64, Cham: Springer International Publishing, 2014.
- [24] Y. An, T. Bell, B. Li, J. Xu, and S. Zhang, “Method for large-range structured light system calibration,” *Applied Optics*, vol. 55, p. 9563, Nov. 2016.
- [25] P.-Y. Kao, S.-W. Shih, Y.-P. Hung, and A. M. Tun, “Recalibration of Structured-Light RGB-D Cameras with Parametric Depth Error Correction,” in *2021 IEEE 4th International Conference on Multimedia Information Processing and Retrieval (MIPR)*, pp. 111–117, Sept. 2021.
- [26] A. Teichman, S. Miller, and S. Thrun, “Unsupervised Intrinsic Calibration of Depth Sensors via SLAM,” in *Robotics: Science and Systems IX*, Robotics: Science and Systems Foundation, June 2013.
- [27] R. Avetisyan, M. Willert, S. Ohl, and O. Städt, “Calibration of depth camera arrays,” in *Proceedings of the 13th SIGRAD 2014 Conference of the Swedish Eurographics Chapter*, (Gothenburg, Sweden), pp. 41–48, June 2014.

- [28] N. Kuppuswamy, A. Alspach, A. Uttamchandani, S. Creasey, T. Ikeda, and R. Tedrake, “Soft-bubble grippers for robust and perceptive manipulation,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9917–9924, Oct. 2020.
- [29] P. O’Dowd, “A Robot That Draws and Shades with Tactile Force Feedback Sensed Through a Pencil,” in *Proceedings of EVA London 2019*, July 2019.
- [30] I. A. Sucan, M. Moll, and L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, pp. 72–82, Dec. 2012.
- [31] J. Kuffner and S. LaValle, “RRT-connect: An efficient approach to single-query path planning,” in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, vol. 2, (San Francisco, CA, USA), pp. 995–1001, IEEE, 2000.
- [32] D. E. Whitney, “Resolved Motion Rate Control of Manipulators and Human Prostheses,” *IEEE Transactions on Man-Machine Systems*, vol. 10, pp. 47–53, June 1969.
- [33] J. Haviland and P. Corke, “NEO: A Novel Expeditious Optimisation Algorithm for Reactive Motion Control of Manipulators,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1043–1050, April 2021, Oct. 2020.
- [34] P. Corke and J. Haviland, “Not your grandmother’s toolbox – the Robotics Toolbox reinvented for Python,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11357–11363, May 2021.

APPENDIX A

APPENDIX

A.1 Technical Drawings for the Gripper Attachment

Figures A.1, A.2 and A.3 are technical drawings illustrating various parts of the gripper attachment. Please note that the scale markings are not accurate since the A4-sized drawings were resized to fit within the margins.

A.2 Additional Data from the Quantifying Pose Estimation Accuracy Experiment

In this section we present the pose estimate comparison data for the ArUco marker estimator, the YOLO estimator and the large ArUco estimator as explained in Section 4.5. Figures A.4 and A.5 are plots comparing the estimator submodules with the large ArUco estimator, and Table A.1 contains the data for Figure A.4 in table form.

Distance to board (m)	ArUco vs. Large ArUco (rad)		YOLO vs. Large ArUco (rad)		YOLO vs. Depth (rad)	
	mean	std. dev.	mean	std. dev.	mean	std. dev.
5	0.224	0.042	0.091	0.023	0.205	0.058
4	0.309	0.034	0.282	0.033	0.072	0.038
3	0.187	0.065	0.232	0.073	0.055	0.014
2	0.061	0.026	0.124	0.031	0.076	0.008

Table A.1: Pose estimate angular differences between the YOLO submodule, the ArUco submodule, and the large ArUco submodule at differing distances.

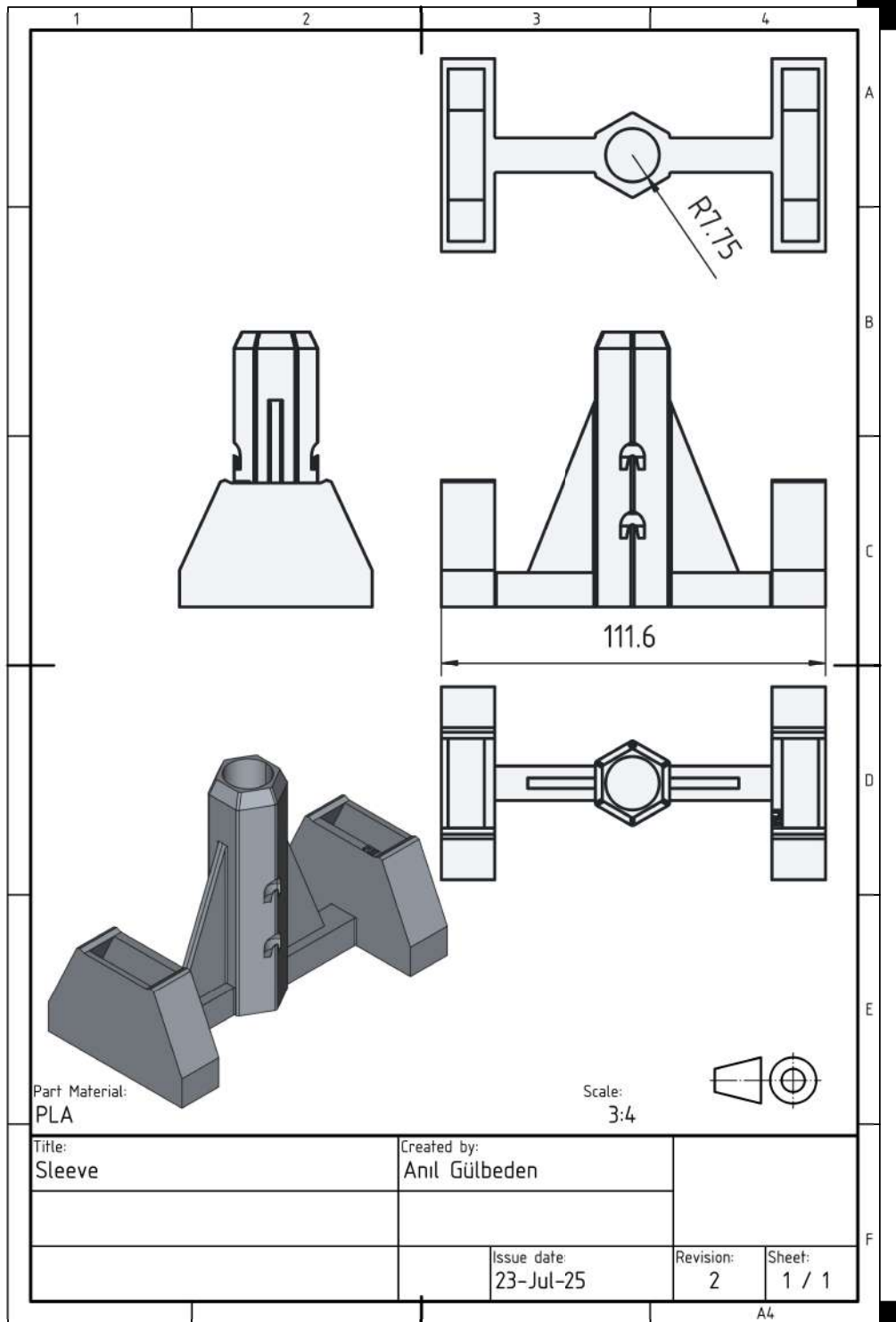


Figure A.1: Sleeve Technical Drawing

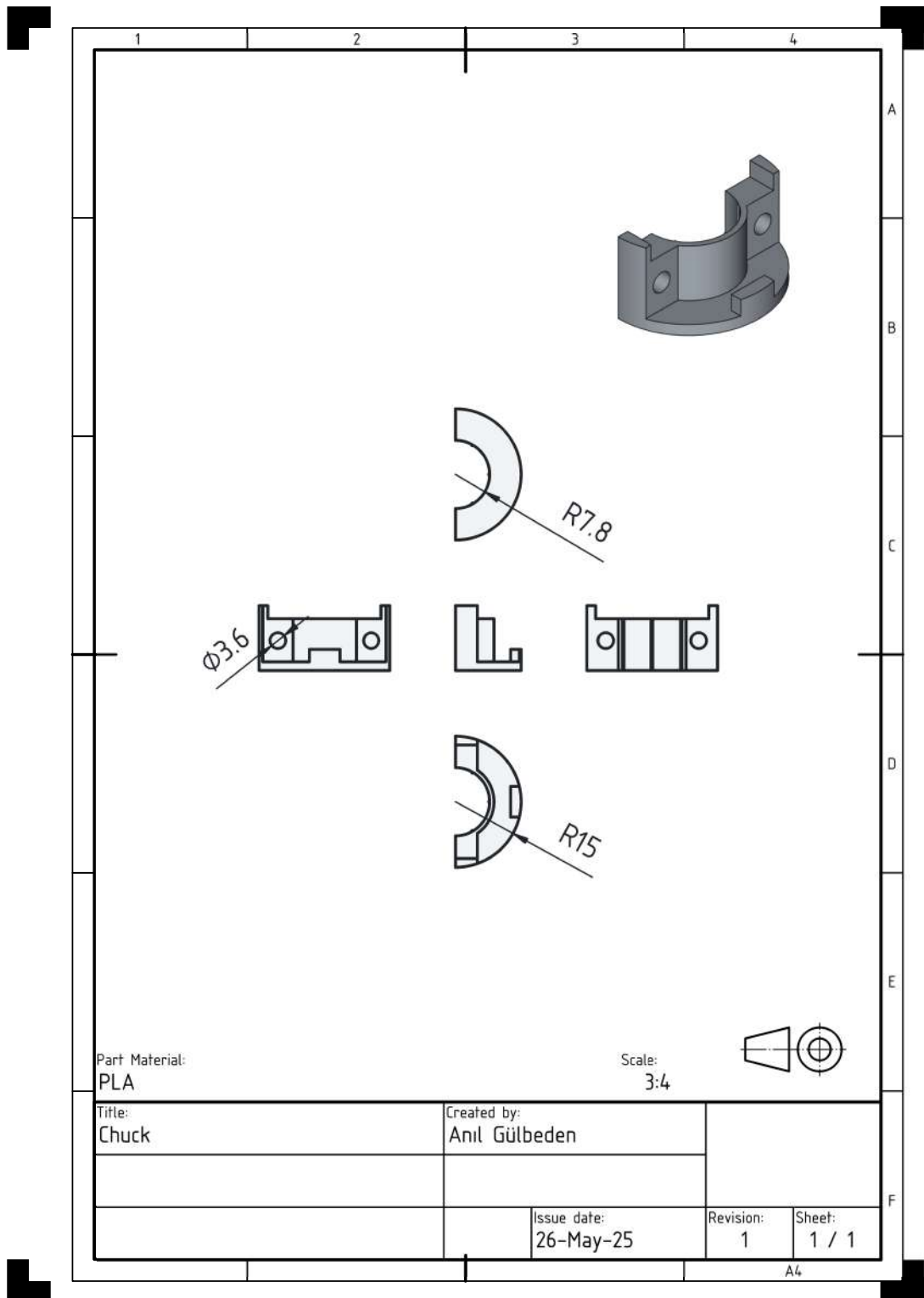


Figure A.2: Chuck Technical Drawing

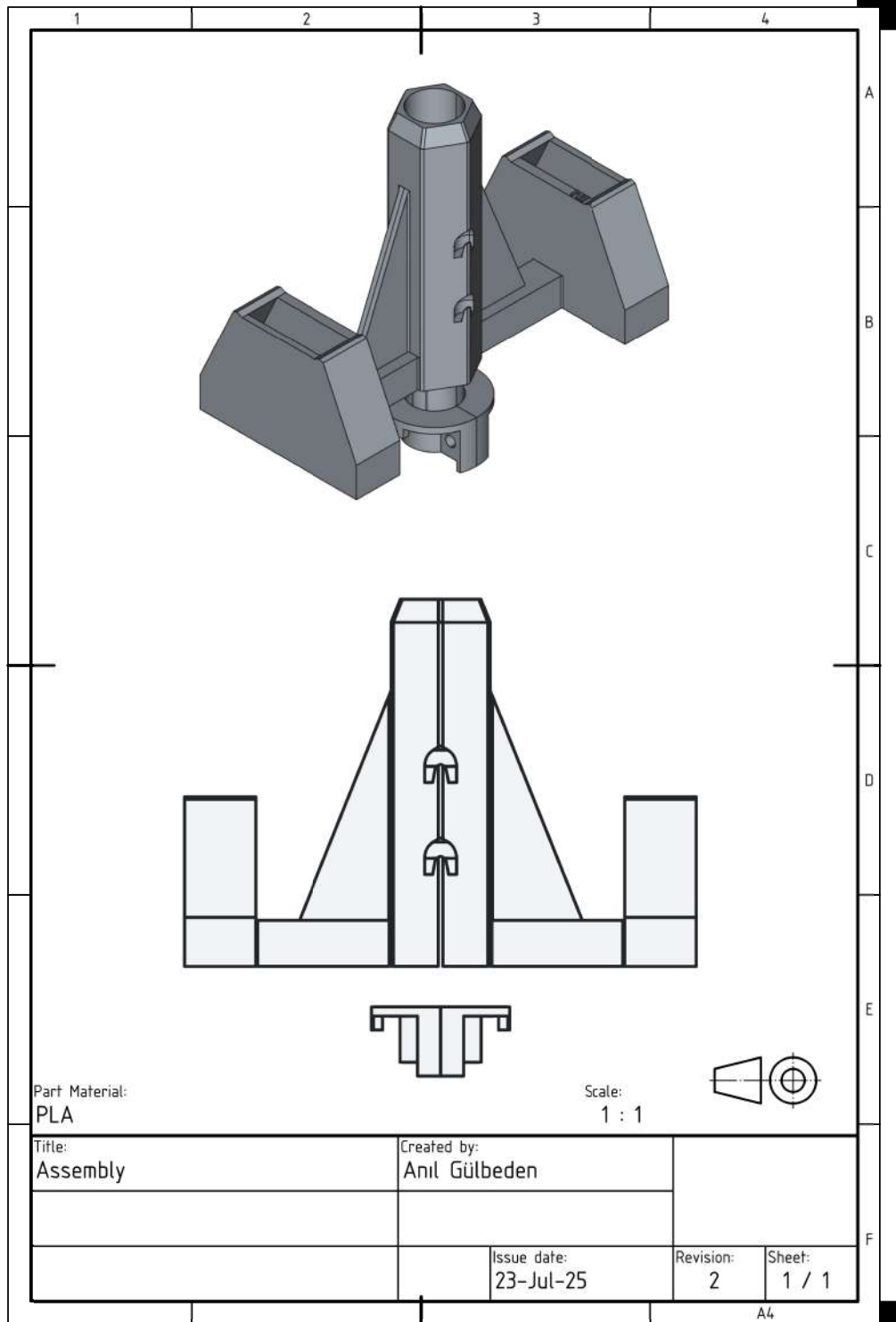


Figure A.3: Gripper Attachment Assembly Technical Drawing

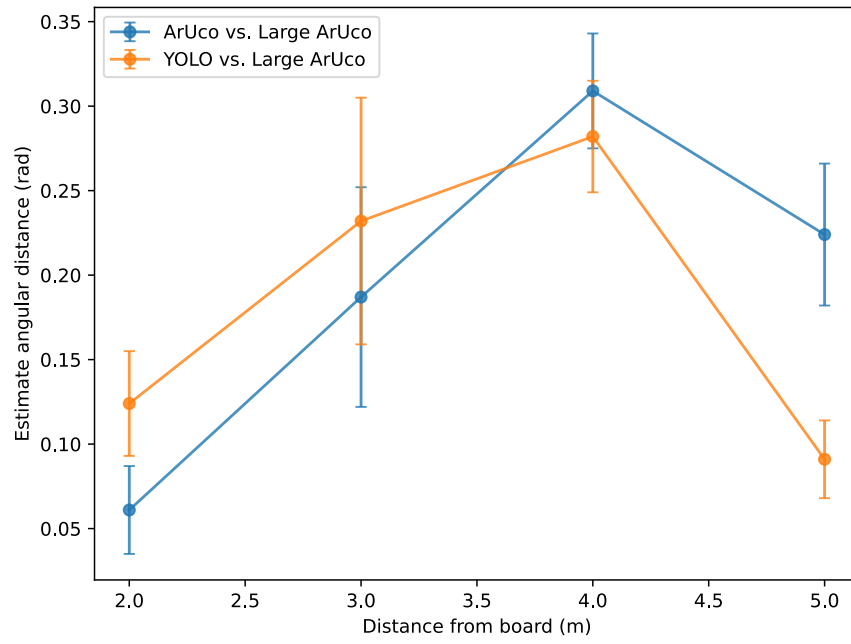


Figure A.4: Board pose estimate angular differences of the YOLO and ArUco submodules in comparison to the large ArUco submodule.

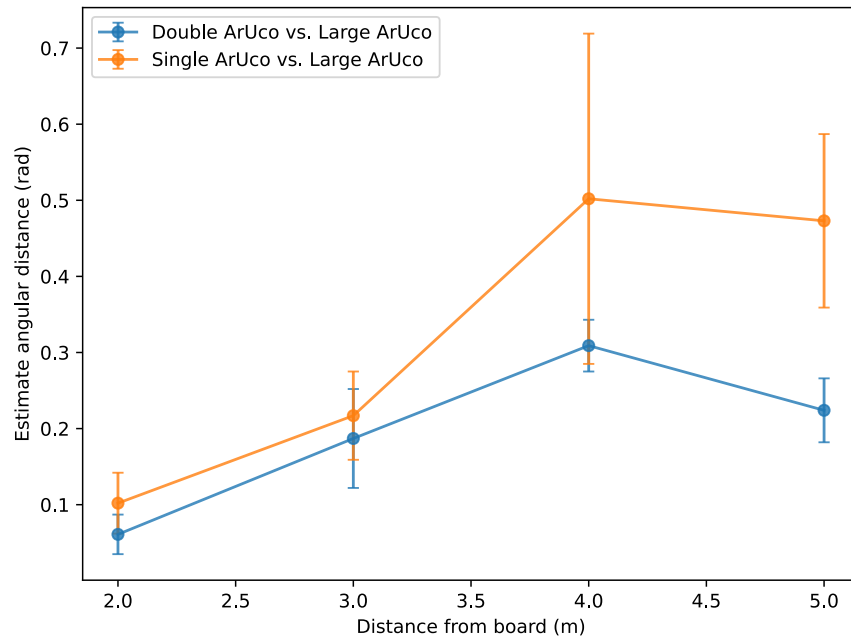


Figure A.5: Board pose estimate angular differences of the ArUco submodules in the single marker and double marker cases in comparison to the large ArUco submodule.